

FEATURE & PAGE INVENTORY AUDIT

Apna Baithak Full Inventory Audit

Exhaustive catalog of every page, feature, API, schema and integration

This document catalogs every customer-side and admin-side page, view, component, API endpoint, database table, third-party integration and cross-cutting system found in

12

CUSTOMER VIEWS

5

ADMIN TABS

46

API ENDPOINTS

9

DB TABLES

8+

3RD-PARTY INTEGRATIONS

180+

DISTINCT FEATURES

Repository: github.com/uutkarssh/Apna-Baithak-Rest

Stack: Next.js 16 · Turso (libSQL) · Supabase (Auth + Storage) · Telegram Bot · Gemini Vision

Audit scope: Customer PWA + Admin PWA (separate manifests) + Backend + Cross-cutting systems

Table of Contents

Table of Contents	2
1. Executive Summary	5
2. Customer-Side PWA	7
2.1 App Shell & View Routing	7
2.2 Home View	8
2.3 Categories View (Menu + Search)	8
2.4 Category Listing View	9
2.5 Item Detail View	9
2.6 Cart View	10
2.7 Checkout View	11
2.8 Orders View	11
2.9 Order Confirmation View	12
2.10 UPI Payment View	13
2.11 Login View	13
2.12 Profile View	14
2.13 Location View (Lazy Wrapper)	15
2.14 Location View Inner (Map + Address Management)	15
2.15 Customer-Side Shared Components	16
3. Admin-Side PWA	19
3.1 Admin Route Page	19
3.2 Admin Layout	19
3.3 AdminAuthProvider	19
3.4 AdminLogin	20
3.5 AdminDashboard (Primary Hub)	20
3.6 Orders Tab (sub-component)	20
3.7 Menu Tab (sub-component)	21

3.8 OrderDetailModal	22
3.9 FeaturedTab	23
3.10 ReviewsPanel	24
3.11 SalesAnalytics	24
3.12 Admin PWA Components	25
4. Backend & API Inventory	26
4.1 Customer-Facing Endpoints	27
4.2 Admin Endpoints	28
4.3 Telegram Webhook Details	30
4.4 UPI Payment State Machine	31
4.5 Pricing Engine (in checkout)	31
4.6 Endpoint Summary Count	32
5. Database Schema Inventory	33
5.1 Category (categories)	33
5.2 MenuItem (menu_items)	33
5.3 MenuItemImage (menu_item_images)	34
5.4 Customer (customers)	34
5.5 Address (addresses)	34
5.6 Order (orders) — CENTRAL TABLE	34
5.7 OrderItem (order_items)	35
5.8 OrderStatusHistory (order_status_history)	35
5.9 PaymentAttempt (payment_attempts)	36
5.10 RestaurantConfig (restaurant_config) — SINGLETON	36
6. Third-Party Integrations	37
6.1 Turso (libSQL)	37
6.2 Supabase Auth (Customer Authentication)	37
6.3 Supabase Storage (Image Storage)	37

6.4 Telegram Bot API (Admin Notifications + Inline Admin Actions)	37
6.5 Google Gemini Vision (UPI Screenshot Verification)	38
6.6 OpenStreetMap Nominatim (Geocoding)	38
6.7 Leaflet + react-leaflet (Interactive Map)	38
6.8 Google Maps (Deep-Links Only — Not API)	38
6.9 Other Notable Dependencies	38
7. Cross-Cutting Systems	40
7.1 Authentication & Session Handling	40
7.2 PWA Features (Customer + Admin separately)	41
7.3 Notification Systems	42
7.4 Real-Time / Live-Updating Features	42
7.5 Image & Media Handling	43
7.6 Admin Controls / App-Wide Toggles	43
7.7 Pricing / Business-Logic Engine	44
7.8 Responsive / Multi-Device Handling	44
7.9 Drag-and-Drop / Advanced UI Interactions	45
7.10 AI / Automation Features	45
7.11 Zustand Stores (Client State)	46
7.12 Scripts Directory (Maintenance Utilities)	46
8. Full Flat Feature List	48

1. Executive Summary

This audit catalogs the complete feature and page inventory of the Apna Baithak restaurant Progressive Web App. The application is built on Next.js 16 with a Turso (libSQL) database, Supabase for customer authentication and image storage, a Telegram Bot for admin notifications and inline admin actions, and Google Gemini Vision for UPI payment screenshot verification. The codebase is a single Next.js project that ships two separately-installable Progressive Web App experiences: a customer-facing PWA mounted at the root route, and an admin PWA mounted at /admin with its own manifest, service worker, and offline fallback page.

Every page, view, component, API endpoint, database table, third-party integration, and cross-cutting system identified in the repository has been enumerated below. The totals at the end of this section reflect the deduplicated count of distinct features available for development-cost estimation. The audit deliberately ignores bugs, code smells, and security observations; the only goal is a complete and accurate count of capabilities.

Top-Level Counts

Dimension	Count
Customer-side pages/views	12 distinct views (single-page app shell)
Admin-side pages/tabs	1 route with 5 dashboard tabs + 4 modal editors
Customer-side React components	29 (incl. shadcn/ui primitives, apna-specific, PWA, providers)
Admin-side React components	9 (auth, login, dashboard, modals, panels, PWA)
Distinct API endpoints	46 method+path combinations across 35 route files
Customer-facing endpoints	22 across 18 files
Admin endpoints	24 across 17 files
Database tables	9 (Category, MenuItem, MenuItemImage, Customer, Address, Order, OrderItem, OrderStatusHistory, PaymentAttempt, RestaurantConfig)
Third-party integrations	8 (Turso, Supabase Auth, Supabase Storage, Telegram Bot, Gemini Vision, OpenStreetMap Nominatim, Leaflet maps, Google Maps deep-links)
Zustand client stores	4 (cart, favorites, app navigation, recent searches)
PWA manifests	2 (customer /manifest.json, admin /admin-manifest.json)
Service workers	2 (/sw.js scope /, /admin-sw.js scope /admin/)
Distinct UI states catalogued	120+ across all pages (loading, empty, error, paused, blocked, etc.)
Total deduplicated features	180+ (see § 8 Full Flat Feature List)

Repository At A Glance

The application is built as a single Next.js 16 codebase using the App Router, React 19, TypeScript, Tailwind v4, and shadcn/ui (new-york style). State management on the client is handled by Zustand with the persist middleware (localStorage). Server-side data operations use Prisma 6 with the @prisma/adapters-libsql driver against Turso. Customer authentication uses Supabase Auth via @supabase/ssr for cookie-persisted sessions. Admin authentication is hardcoded against environment variables with a single httpOnly cookie (ab_admin) — no database-backed session.

The customer PWA has a single-page-app architecture: a root client component (ApnaBaithakApp) maintains an internal view stack in a Zustand store, and swaps between 12 distinct views (Home, Categories, Category Listing, Item Detail, Cart, Checkout, Orders, Order Confirmation, UPI Payment, Login, Profile, Location). Navigation is integrated with the browser history API so the Android/iOS hardware back button walks the in-app stack before leaving the website. The admin PWA is a single page mounted at /admin, gated by a client-side auth context, with a tab bar that switches between Orders, Menu, Featured, Analytics, and Reviews panels.

Backend Shape

The backend consists of 35 Next.js Route Handler files exposing 46 distinct method+path combinations under /api. Customer-facing routes (/api/menu, /api/checkout, /api/orders, /api/addresses, /api/settings, /api/auth/me, /api/ratings, /api/telegram/webhook) are mostly public or protected by Supabase JWT. Admin routes (/api/admin/*) are guarded by a single boolean cookie check (ab_admin === '1'). The Telegram webhook (/api/telegram/webhook) is the secondary admin surface — it accepts callback_query events from inline keyboard buttons and performs verify/reject actions on pending UPI payments.

2. Customer-Side PWA

The customer PWA is a single-page Next.js application. The root route (/) renders a server component that mounts the ApnaBaithakApp client component, which contains the entire customer experience: a state-machine-driven view router, browser history integration, persistent cart/favorites/selected-address via localStorage, and 12 distinct views that swap with Framer Motion animations. Authentication is soft-gated — browsing the menu, building a cart, and searching are available without login; only account-scoped actions (favorites, checkout, address saving, viewing orders) require sign-in.

2.1 App Shell & View Routing

File: `src/components/apna/apna-baithak-app.tsx`

Purpose: Top-level SPA shell — mounts TopNav, TopBar, the active view, StickyCartBar, and BottomNav.

Features:

- useMounted hook via useSyncExternalStore — returns false during SSR, true on client; suppresses cart-dependent UI until hydration completes.
- Framer Motion AnimatePresence view-switch animation: opacity + y-translate (initial opacity:0 y:8 → animate opacity:1 y:0 → exit opacity:0 y:-4), 180ms easeOut.
- View-routing state machine reads view from useApp Zustand store; selects among home, categories, category-listing, item-detail, cart, location, profile, orders, checkout, login, upi-payment, order-confirmation.
- upi-payment and order-confirmation views gated on activeOrder being truthy (render nothing if absent).
- TopBar visibility logic: shown only on the home view (hidden on item-detail, cart, category-listing, categories, location, checkout, orders, profile, login, upi-payment, order-confirmation).
- StickyCartBar visibility logic: shown when mounted AND cart count > 0 AND view is one of home, categories, category-listing, item-detail.
- TopNav (landscape) always rendered; CSS hides it on portrait via landscape:hidden.
- BottomNav (portrait) always rendered; CSS hides it on landscape.
- 4px bottom spacer div when cart has items (portrait only) — reserves space below the floating cart bar so it doesn't cover content.
- Browser history integration: on mount, replaces current history entry with {apnaBaithak:true, view:'home'} (guarded by historyReady ref). On every subsequent view change (when not consuming a popstate), pushes {apnaBaithak:true, view} onto the history stack.
- popstate listener: if event state has apnaBaithak:true, calls back() from the app store (and sets skipHistoryPush so the next view-change effect doesn't double-push). Hardware back button on Android/iOS therefore walks the in-app stack before leaving the website.
- Scroll-to-top on view change: on each view change, requests an animation frame and resets window.scrollTo, documentElement.scrollTo, body.scrollTo, and scrollTop to 0.
- Deep-link / query-param handling: on mount (and when openItem reference changes), reads ?item= from window.location.search. If present, calls openItem(itemSlug) (switches view to item-detail), then strips the ?item= param via history.replaceState.
- suppressHydrationWarning on root and main — tolerates Zustand persist hydration mismatches.

Conditional / dynamic behavior:

- hideTopBar derived from view ∈ HIDE_TOPBAR_VIEWS.
- showCartBar derived from mounted AND count > 0 AND view ∈ CART_BAR_VIEWS.

- 4px bottom spacer when count > 0 AND mounted (portrait only via landscape:hidden).
- Responsive padding on : px-0 on mobile, sm:px-6, lg:px-8; pb-24 portrait, pb-6 landscape.

2.2 Home View

File: src/components/apna/views/home-view.tsx

Purpose: Customer landing page — search entry, category rail, hero promo card, featured items rail, popular items grid, restaurant info footer.

Features:

- Search entry button (not a real input — navigates to categories view, where the actual search input is autofocused).
- Live order-acceptance polling: fetches /api/settings with refetchInterval 30_000ms (every 30 seconds). When isAcceptingOrders === false, shows a calm blue 'We're currently closed' banner at the top of home.
- Categories rail: horizontal scroll on mobile (.no-scrollbar), responsive grid (sm:4 / md:6 / lg:8 cols) on larger screens. Each CategoryButton shows category image or UtensilsCrossed fallback icon + name; click calls openCategory(slug) which navigates to category-listing view.
- Category loading skeletons: CategorySkeleton × 8 (different layout for mobile rail vs desktop grid).
- Hero promo card with gradient from-brand to-red-700, decorative circles, RESTAURANT.name, headline 'Freshly made. Delivered within 10 km.', delivery ETA chip, 10-km radius chip.
- Average rating chip on hero (only if avgRating > 0) with star icon and total review count — fetched from /api/ratings.
- Featured items section: fetches /api/menu/featured; horizontal scroll rail of ItemCards (width 160px each) on mobile; responsive grid (sm:2 / md:3 / lg:4 / xl:5 / 2xl:6) on larger screens. Section entirely hidden if featured.length === 0 (graceful fallback). Shows count badge next to heading. 'See all →' button navigates to categories.
- Featured items loading skeleton: ItemCardSkeleton × 4 (mobile rail) or × 8 (desktop grid).
- Popular items section: always shown, displays items.slice(6, 20) as a responsive grid (2/3/4/5/6 cols). Count badge next to heading. Loading skeleton × 6.
- Restaurant info footer card with MapPin avatar, BrandWordmark image, full address, phone number.
- CategoryButton sub-component: motion.button with scale-in animation (delay index × 0.05 capped at 0.4s); circle with category icon, label with line-clamp-1.

UI States:

- Loading (categories + items) — skeleton placeholders.
- Loaded with content — full home view.
- Loaded, orders paused — calm blue closed banner at top.
- Loaded, no featured items — Featured section hidden entirely.
- Loaded, rating unavailable (avgRating === 0) — rating chip hidden.

2.3 Categories View (Menu + Search)

File: src/components/apna/views/categories-view.tsx

Purpose: Main Menu screen with live search, recent searches, popular searches, and category grid.

Features:

- Back button (round, muted bg) calling back() from app store.
- Sticky header (z-10, blurred bg) containing back button, 'Menu' heading, and pill-shaped search input with autoFocus.

- Search input with clear (X) button when query non-empty.
- Live search: calls `/api/menu/items?search=` via `authedFetch` (query enabled only when `search.trim()` is non-empty).
- Recent searches section: pills from `useRecentSearch` Zustand store, each click re-runs the search. 'Clear' button (trash icon) wipes the list. `AnimatePresence` for pill enter/exit.
- Popular searches section: static pill list — ['Pizza','Burger','Pasta','Maggie','Chaat','Fries'] — click runs the search.
- Search-results mode: when search query non-empty, hides categories grid and shows 'N result(s) for ' header; renders `ItemRows` in a responsive grid (1/2/3/4 cols).
- Empty-search-results state: dashed bordered card 'No items match ' with hint to try pizza/burger/chaat.
- Categories grid (default mode): 2/3/4/6-col responsive grid of category cards with image (or `UtensilsCrossed` fallback), name, and `itemCount` ('N items' or 'N item'); click calls `openCategory(slug)`.
- Auto-record recent searches: `useEffect` records the current search query into `useRecentSearch.add()` if `search.trim().length >= 2` AND `itemsData.items.length > 0` AND `lastRecorded.current !== search` (so the same query isn't recorded twice in a row).
- `runSearch(q)` helper: calls `setSearch(q)`.

2.4 Category Listing View

File: `src/components/apna/views/category-listing-view.tsx`

Purpose: List all menu items within a single category.

- Sticky header with back button and category name (resolved from `activeCategorySlug` via the categories query). Item-count badge (e.g. '12 items').
- Category description paragraph (only if `category.description` exists).
- Items list: responsive grid (1/2/3/4 cols) of `ItemRows` fetched from `/api/menu/items?category=`.
- Loading skeleton: `4 × ItemRowSkeleton`.
- Empty state: `motion.div` with Package icon and 'No items in this category yet / Please check back soon.'
- `useQuery` enabled only when slug is truthy.

2.5 Item Detail View

File: `src/components/apna/views/item-detail-view.tsx`

Purpose: Full detail page for a single menu item with gallery, qty picker, add-to-cart, related items.

- Hero image area: `ImageGallery` (single image OR multi-image carousel) — falls back to `UtensilsCrossed` icon when no images.
- Top gradient overlay for button legibility.
- Back button (top-left, white circle, blurred bg) — calls `back()`.
- Share button (top-right): tries `navigator.share({title, text})` first; falls back to `navigator.clipboard.writeText('/?item=') + toast 'Link copied'` if share API unavailable.
- Content card slides up over the hero (`-mt-6 rounded-t-3xl`).
- Veg indicator (square border + dot, `emerald-600`) if `item.isVeg`.
- Category label (uppercase, tracked, muted) if `item.category`.
- Item name (`h1`, `2xl` extrabold).
- Inline qty selector — pill with Minus button (rounded white, brand text), qty display, Plus button (brand bg, brand-foreground). Local qty state starts at 1; min 1 (Minus disabled at 1).
- Price (`2xl` extrabold brand color) with 'per piece' suffix.
- Star rating badge (amber) if `item.rating > 0`.

- Prep-time badge (blue) if item.prepTimeMins — shows 'X-(X+10) mins'.
- Calories badge (orange) if item.calories — shows 'N kcal' with Flame icon.
- Description section ('About this dish') if item.description.
- Restaurant note card (brand-soften bg, Store icon) — 'Prepared fresh at . Orders cannot be cancelled once preparation begins.'
- Related items rail ('More from ') — horizontal scroll of up to 6 related items (excluding current) from the same category; each is a 128px-wide card with image, name, price; clicking calls openItem(r.slug) and resets local qty to 1.
- Sticky bottom bar (z-20, blurred bg, pb-[calc(0.75rem+env(safe-area-inset-bottom))]): left-side total preview ('N items · Total '), right-side primary CTA.
- CTA label logic: if item already in cart → 'Add N more · ' (calls setQty(item.id, inCart.quantity + qty) then navigates to cart); else → 'Add to cart' (calls addItem(item, qty) then navigates to cart).
- Toast on add/update: '\$ {qty} × added to cart' / 'Updated quantity'.

UI States: Loading / no item — three-block shimmer (title, price, paragraph). Loaded — full detail layout. CTA label switches between 'Add to cart' and 'Add N more' based on inCart presence.

2.6 Cart View

File: src/components/apna/views/cart-view.tsx

Purpose: Cart line items + address confirmation + live delivery-fee calculation + sticky checkout bar.

- Sticky header with back button, 'Your Cart' title, and item-count badge (lines.reduce(qty)).
- Empty-cart state: dashed bordered centered card with ShoppingBag icon, 'Your cart is empty', helper text, 'Browse menu →' CTA (goes home), and a secondary 'Or search for a specific dish' link (goes to categories).
- Address confirmation card (Instamart-style): brand-soften bg, MapPin avatar, 'Delivering to' / 'Add address to proceed' label, full address line, distance ('X km away'), 'Change' / arrow button; clicking goes to location view (or goToLogin('cart') if not authed).
- Cart line items list: for each line — image (or UtensilsCrossed), name, '₹X each', 'Remove' button (red text), QtyStepper (sm), line total rupees(price*qty).
- '+ Add more items' link (brand color) — navigates home.
- Live delivery-pricing message card: dynamic color/content based on state — emerald when free delivery unlocked (subtotal ≥ ₹2000), amber when blocked (below ₹200 OR below ₹800 if >7km), brand-soften when eligible with progress to free delivery.
- No-address-selected state: muted card 'Select a delivery address to see the delivery charge.'
- Bill Details card: Item Total / Delivery Fee (or 'Select address' / 'FREE' if free) / To Pay (lg extrabold brand color).
- Non-cancellable note (amber card, ShieldCheck icon) — 'Orders cannot be cancelled or refunded once preparation begins. By placing an order you agree to Apna Baithak's cancellation & refund policy.'
- Sticky checkout bar (z-20): left side 'Delivering to , ' (or 'Select address') and ' · N items'; right side 'Place Order' (or 'Sign in to order' if not authed) with ArrowRight.
- checkout() function logic: if authLoading return; if no profile → goToLogin('cart'); if no chosen address → setView('location'); else → setView('checkout').
- Delivery fee calculation mirrors the server: deliveryChargeForDistance, checkOrderEligibility, remainingForFreeDelivery, FREE_DELIVERY_OVERRIDE, MIN_ORDER_SUBTOTAL, FAR_DISTANCE_THRESHOLD_KM, FAR_MIN_ORDER_SUBTOTAL.

UI States: empty cart; cart with items, no address; cart with items, address known, eligible; cart with items, address known, blocked (below min); cart with items, address known, free delivery unlocked. Checkout button label 'Place Order' vs 'Sign in to order' by auth state.

2.7 Checkout View

File: `src/components/apna/views/checkout-view.tsx`

Purpose: Final order placement — address, payment mode, notes, bill, place-order call.

- Sticky header with back button and 'Checkout' title.
- Phone-required guard: on mount (and whenever profile changes), if profile has no phone, sets `localStorage` flag `apna-baithak-require-phone=1`, shows error toast 'Contact number is required before checkout', and navigates to profile view (which auto-opens the Edit Profile modal).
- Delivery Address section: chosen address card (label, full address, distance) with 'Change' button; or dashed 'Add a delivery address to proceed' prompt if no chosen address.
- Payment Mode section: two selectable cards (COD with Banknote icon, UPI with CreditCard icon) — each shows label + description + radio-style indicator (filled brand circle with `CheckCircle2` when selected). Sourced from `PAYMENT_MODES` constant.
- Order Notes section: optional textarea ('e.g. Less spicy, call on arrival...').
- Bill Details card: Item Total (N items) / Delivery Fee (or 'Select address' / 'FREE') / To Pay.
- Calm 'We'll be back soon' banner when `!isAcceptingOrders` (orders paused) — blue card with Clock icon. Takes priority over the eligibility block banner.
- Hard-block banner when order is ineligible AND orders are accepting — amber 'Checkout blocked' card with `blockMessage` (from `eligibility.message`).
- Policy note ('By placing this order you agree that orders cannot be cancelled or refunded once preparation begins at .').
- Sticky Place Order button (z-20, with safe-area bottom padding). Disabled if `placing || authLoading || !profile.phone?.trim() || !chosen || lines.length === 0 || isBlocked || !isAcceptingOrders`.
- Button label by state: 'Placing order...' with `Loader2` spin while placing; 'Orders paused' when `!isAcceptingOrders`; 'Checkout blocked' when `isBlocked`; 'Pay ☒ X' when UPI mode; 'Place Order ☒ X' otherwise.
- `placeOrder()` function: re-validates phone, address, cart emptiness; POST `/api/checkout` with `{ items: [{itemId, quantity}], addressId, paymentMode, notes }`.
- On 503 / `ORDERS_PAUSED` error: silently invalidates `restaurant-settings` query (so banner appears) without showing a toast.
- On success: if UPI → `goToUpiPayment(order)` + toast 'Order placed! Complete your UPI payment.'; if COD → `clear() cart` + toast 'Order placed!' + `goToOrderConfirmation(order, 'PAID')`.
- Order-acceptance polling: `useQuery` for `/api/settings` with `refetchInterval 30_000ms`.

UI States: loading (auth); no phone (redirect-to-profile triggered); no address (Add a delivery address prompt); eligible, accepting orders (normal checkout); eligible, paused (calm blue banner, button disabled as 'Orders paused'); ineligible (amber block banner, button disabled as 'Checkout blocked'); placing order (spinner + 'Placing order...').

2.8 Orders View

File: `src/components/apna/views/orders-view.tsx`

Purpose: Customer's order history with expandable tracking cards, reorder, receipt download, and rating.

- Sticky header with back button and 'My Orders' title.
- Loading state: `2 × OrderCardSkeleton`.
- Empty state: dashed centered card with Package icon, 'No orders yet', 'Your placed orders will appear here.', and 'Start ordering' CTA (goes home).
- Order list: each order rendered by `OrderTrackingCard`.
- Bottom 80px spacer when cart has items (avoids sticky cart bar covering content).

- OrderTrackingCard: expandable header (click toggles expanded): icon circle (red for cancelled, emerald for delivered, brand-softener otherwise), order number, StatusBadge, item-count + formatted datetime, total amount + payment mode.
- Status colors: NEW=blue, PREPARING=amber, OUT_FOR_DELIVERY=purple, DELIVERED=emerald, CANCELLED=red.
- Live tracking timeline (only for active orders — not cancelled/delivered): 4-step horizontal timeline (NEW → PREPARING → OUT_FOR_DELIVERY → DELIVERED) with connecting lines that animate width 0→100% when step is done, an animated pulse ring around the active step's circle, step labels, and the active step's description below.
- ETA estimate: $\sim \max(0, 35 - \text{elapsedMin})$ min badge in the timeline header (re-computed on every render — but there's no auto-poll of orders in this view).
- Delivered state — emerald bar 'Delivered on '.
- Cancelled state — red bar 'This order was cancelled.'
- Expanded details section (AnimatePresence height animation): items list, delivery address block (MapPin icon + addressLine + distance), bill summary (Item Total / Delivery Fee / Total Paid), action buttons row (Call tel: link, Reorder, Receipt download).
- Reorder: iterates order.items, calls addItem(...) for each (skipping items with null itemId — deleted menu items), then navigates to cart; toast '\${added} items from added to cart' or 'These items are no longer on the menu' if zero added. Disabled when reordering or cancelled.
- Receipt download: fetches /api/orders/<id>/receipt as a blob, creates an object URL, programmatically clicks a hidden <a download='receipt-<number>.pdf'>, then revokes URL after 60s. Uses fetch(credentials:include) rather than window.open() to avoid mobile popup-blocking. Disabled when not delivered; shows 'Receipt available after delivery' tooltip when disabled; spinner via animate-bounce while downloading.
- Rating section (only for delivered orders): if order.rating != null — display amber card 'Your Rating:' + 5 stars + optional review text; if no rating yet — interactive OrderRating component.
- Fetches orders via authedFetch('/api/orders'); query enabled only when !authLoading && !!profile.

2.9 Order Confirmation View

File: src/components/apna/views/order-confirmation-view.tsx

Purpose: Post-checkout success / pending-verification screen.

- Clears cart on mount via useEffect(() => clearCart(), [clearCart]).
- Sticky header with back button that goes directly to home (NOT back to checkout/payment — the goToOrderConfirmation action trims 'checkout' and 'upi-payment' from the nav stack).
- Status hero: PENDING_VERIFICATION — amber circle with AlertTriangle icon, 'Payment Under Review' heading; PAID — emerald circle with CheckCircle2 icon, 'Order Confirmed!' heading.
- Order Summary card: Items count, Amount (lg extrabold brand color), Payment badge (Pending → amber 'Pending Verification', Paid+UPI → emerald 'Paid (UPI)', COD → emerald 'Cash on Delivery'), Distance row (if order.distanceKm != null).
- Delivery Address card: full address line, distance + ETA ('X km from restaurant · ETA ~25-35 min').
- Pending verification info box (amber): 'What happens next?' with 4 bullet points (manual review of screenshot, confirmation in a few minutes during business hours, track from Orders page, we'll call if there's an issue).
- Action buttons: 'Track my order' (brand bg) → goes to orders view; 'Back to home' (outlined, HomeIcon) → goes to home view.
- Help footer: 'Need help? Call us at or email '.

UI States: Paid (COD success or UPI verified) — emerald hero, 'Order Confirmed!'. Pending verification (UPI under manual review) — amber hero, 'Payment Under Review', extra info box.

2.10 UPI Payment View

File: `src/components/apna/views/upi-payment-view.tsx`

Purpose: UPI payment attempt lifecycle: check status, initiate, show QR + countdown + upload, AI-verify screenshot, expire + retry.

- Order summary card: order number + amount to pay (2xl extrabold brand color).
- Five-phase state machine: checking (initial status check), initiating (creating new attempt), awaiting_payment (QR + Pay button + countdown + upload visible), verifying (screenshot uploaded, Gemini AI verifying — overlay state within awaiting_payment render), expired (5-min window lapsed, retry available).
- Mount guard: calls `/api/orders//upi/status`; if order is PAID → `goToOrderConfirmation(order, 'PAID')`; if PENDING_VERIFICATION → `goToOrderConfirmation(order, 'PENDING_VERIFICATION')`; else → `initiateAttempt()`.
- `initiateAttempt()`: POST `/api/orders//upi/initiate` → stores attempt, deepLink, qrDataUrl; sets phase to awaiting_payment; auto-opens UPI app via `window.location.href = data.deepLink` (will fail silently on desktop, where QR is primary).
- Countdown timer: every second computes `remainingMs = attempt.expiresAt - Date.now()`; switches to expired phase when 0.
- Time formatting: MM:SS (zero-padded).
- Countdown banner: red bg when `remainingMs < 60000`, amber bg otherwise; shows 'Time remaining: MM:SS' or 'Verifying payment...' when verifying.
- QR code card: 256×256 image of qrDataUrl, payee name + VPA, UPI transaction ref.
- 'Pay ☒ via UPI app' button — re-opens deep link via `window.location.href = deepLink`. Disabled while verifying.
- Upload section: GPay-specific guidance (blue card — screenshots are blocked in Google Pay, use Share button or Transaction History); PhonePe/Paytm/BHIM guidance (emerald card); hidden ; `handleUpload(file)` validates image type and ≤5MB size; POST `/api/orders//upi/upload` with `FormData(screenshot, attemptId)`.
- On upload outcome VERIFIED: toast success + `goToOrderConfirmation(order, 'PAID')`.
- On upload outcome PENDING_VERIFICATION: toast + `goToOrderConfirmation(order, 'PENDING_VERIFICATION')`.
- Upload button label switches to 'Verifying with AI...' with spinner during verify.
- 'I have paid — Continue without screenshot' secondary button (amber border): calls `confirm()` dialog explaining manual verification; POST `/api/orders//upi/manual` with `attemptId`; auto-redirects to order-confirmation (pending verification state).
- Helper note at bottom: 'Once you upload the image, we'll verify it and take you to your order confirmation automatically. If we can't verify it instantly, your order is still placed — our team will confirm manually.'
- Expired state: centered red X icon, 'Payment Window Expired' heading, 'Start new payment attempt' button (calls `initiateAttempt()`).
- Policy footer: 'By paying, you agree that orders cannot be cancelled or refunded once preparation begins at . If you face any issues, call us at .'
- Header back button: cancels and goes directly to orders view (not back to checkout).

UI States: checking — spinner; initiating — spinner; awaiting payment — countdown + QR + Pay button + Upload + Continue-without-screenshot; verifying — countdown shows 'Verifying payment...', Pay button disabled, Upload button shows 'Verifying with AI...'; expired — retry CTA. Live updates: countdown timer ticks every second; phase transitions drive view re-renders.

2.11 Login View

File: `src/components/apna/views/login-view.tsx`

Purpose: Sign-in / sign-up + Google OAuth + email-verification sub-screen.

- Back button (round muted bg) — calls back().
- Brand icons: BrandIcon (72px) + BrandWordmark (32px) at the top, both priority.
- Heading: 'Welcome back!' (signin) or 'Create your account' (signup).
- Subtitle: 'Sign in to place your order. Browse the menu freely without logging in.'
- Continue with Google button (white card, Chrome icon in brand color, brand text).
- OR divider with horizontal lines.
- Email/password form: in signup mode — Full Name input + Contact Number input (with tel-national autocomplete, numeric inputmode, 12-char max, required, 'Required for delivery — we'll call/SMS on this number.' hint). Email input (email type). Password input (min length 6). Submit button: 'Please wait...' while busy, else 'Sign in' / 'Create account'.
- Indian mobile normalization (normalizeIndianMobile): strips non-digits, strips 91 prefix (12 digits) or 0 prefix (11 digits) to get a clean 10-digit number; validates `^[6-9]\d{9}$`.
- Sign-in flow: `supabase.auth.signInWithPassword({email, password})`; on success toast 'Signed in' and call `returnFromLogin()` (returns to the view the user was bounced from, or cart by default).
- Signup flow: `supabase.auth.signUp({email, password, options:{data:{full_name, phone}}})`; if no session returned (email-confirmation required), shows the verification-sent sub-screen instead of closing the view.
- Verification-sent sub-screen: Back-to-sign-up button, Brand icons, emerald CheckCircle2 (72px), 'Verify your email' heading, 'We have sent a verification email to .' info box, 'Resend verification email' button (calls `supabase.auth.resend({type:'signup', email})`), 'Back to sign in' link.
- Google OAuth calls `signInWithGoogle()` from auth provider (redirects via Supabase OAuth to Google, redirect back to /).
- Mode toggle: 'Don't have an account? Sign up' / 'Already have an account? Sign in'.
- Error handling: toasts error messages on any failure.

UI States: sign-in mode (default); sign-up mode; verification email sent (signup confirmation sub-screen); busy (button disabled, label 'Please wait...' / 'Sending...').

2.12 Profile View

File: `src/components/apna/views/profile-view.tsx`

Purpose: Account hub — avatar, edit profile, saved addresses, payment modes, favorites, recent orders, restaurant info, logout.

- Auto-open EditProfileModal on mount if `localStorage.apna-baithak-require-phone === '1'` (set by checkout view when phone missing). Clears the flag after reading.
- Loading state: centered spinner + 'Loading your profile...'.
- Not-signed-in state: centered UserIcon avatar, 'You're not signed in', 'Sign in to see your orders & saved addresses.', 'Sign in' CTA (calls `goToLogin('profile')`).
- Signed-in layout: Back button (header). Profile section (centered): large round avatar (image if `profile.avatarUrl` else initial letter on brand bg), name/email, phone if set.
- Edit Profile button (brand bg, Pencil icon) and Logout button (outlined, LogOut icon) — logout clears cart, toasts 'Signed out', navigates home.
- Account section: 'Saved Address' row → `setView('location')`; 'Payment Modes' row → `setView('cart')` (lists `PAYMENT_MODES` IDs joined with ' · ').
- Favorites section (FavoritesSection sub-component): for each favorite — image (or UtensilsCrossed), name, price, ADD button (adds to cart + toast), Remove-from-favorites button (red bg, Trash2 icon). `AnimatePresence` with layout animations.
- My Orders section: 'View all →' link if orders exist; shows first 3 orders as compact cards (order number, total, items summary, relative date, status badge). Empty state — dashed card with Receipt icon and 'No orders yet' text.

- Restaurant section: card with restaurant name, full address, phone (tel: link style), email.
- EditProfileModal sub-component (bottom-sheet on mobile, centered dialog on desktop): Full Name input (max 100 chars); Contact Number input with +91 prefix display, 10-digit max, numeric inputmode; Email input (disabled, muted); Indian mobile normalization on save; Validation: name not empty, phone matches `^[6-9]\d{9}$`; Save button → `updateProfile({name, phone:'+'})`; Cancel button; Saving state — spinner + 'Saving...'.

2.13 Location View (Lazy Wrapper)

File: `src/components/apna/views/location-view.tsx`

Purpose: Dynamic-import wrapper for the Leaflet-powered location-view-inner (SSR-disabled, client-only). Features: next/dynamic import of LocationView with `ssr:false`; Loading placeholder while the chunk loads: centered 'Loading map...' text in a 60vh container.

2.14 Location View Inner (Map + Address Management)

File: `src/components/apna/views/location-view-inner.tsx`

Purpose: Leaflet map for pin-drag address selection + saved-address list + add/edit/delete address forms.

- Custom Leaflet markers: `pinIcon` — red SVG pin (34×42) for the customer's draggable location; `restaurantIcon` — black SVG storefront icon (30×36) for the restaurant.
- `DraggablePin` sub-component: Leaflet Marker with `draggable:true`; updates on drag (live) and `dragend` (final) — live distance label updates as the user drags.
- `ClickToMove` sub-component: map-click handler that moves the pin to the clicked `latlng`.
- `Recenter` sub-component: calls `map.flyTo(center, max(currentZoom, 15), {duration:0.6})` whenever the pin position changes (so the map follows the pin).
- Sticky header: back button + 'Select Your Location' title + search input ('Search an area, landmark or pincode').
- Search-as-you-type: `runSearch(q)` calls `geocode(q)` and renders results as a scrollable dropdown (max-height 48 / 192px) with `MapPin` icons; clicking a result sets the pin and replaces the search query with the result label.
- Live distance badge (bottom of map): 'X.XX km from restaurant' + Within/Outside delivery-area chip (emerald `CheckCircle2` 'Within 10 km' or red `AlertTriangle` 'Outside delivery area').
- Re-center floating button (top-right of map): globe/crosshair icon → calls `useCurrentLocation()`.
- Auto-locate on mount: `silent.navigator.geolocation.getCurrentPosition` (high accuracy, 8s timeout) — moves pin to user's location on first mount; silently fails (user can use the button later).
- Reverse geocoding: debounced (500ms) `reverseGeocode(pin)` after pin moves — auto-fills pincode (regex `\b\d{6}\b`) and city (third-from-last comma-separated part) if those fields are empty.
- Action buttons (2-col grid): 'Use Current Location' (re-fetches geolocation) + 'Add New Address' (clears form, scrolls to it).
- Saved Addresses section (sub-component `SavedAddresses`): Loading state — 2 skeleton rows; Empty state — section hidden; List of saved addresses — each card shows label (with 'Default' star badge if applicable), full address, distance, and an action row when not editing: 'Set as default' (hidden if already default), 'Edit' (Pencil), 'Delete' (Trash2). Selected address highlighted with brand border.
- `EditAddressForm` inline (when editing): label selector (Home/Work/Other), House/Flat, Street/Area, Landmark, City, PIN code (numeric, 6-digit max), Save / Cancel buttons.
- `setDefault(id)`: `PATCH /api/addresses/` with `{isDefault:true}`.
- `deleteAddress(id)`: `confirm()` then `DELETE /api/addresses/`.

- Selecting an address calls `onSelect(id) → setSelectedAddressId(id) + setView('cart')`. All mutations invalidate the shared `['addresses']` query cache.
- Address form: Label selector (Home/Work/Other) — pill buttons; House/Flat (required), Street/Area (required), Landmark (optional), City (required), PIN Code (required, numeric only, 6-digit max); 'Outside delivery area' red warning when `!withinRadius`; Submit button disabled when saving || `!withinRadius`; label 'Saving...' while saving.
- `saveAddress()`: validates auth (goes to `goToLogin('location')` if not authed), validates required fields, validates `withinRadius`; `POST /api/addresses` with `{label, houseFlat, streetArea, landmark, city, pincode, latitude, longitude, distanceKm, isDefault:true}`; on success calls `setSelectedAddressId(address.id)`, invalidates `['addresses']` cache, toasts 'Address saved', navigates to cart.

UI States: loading map chunk (delegated to outer wrapper); loading saved addresses (skeletons); geolocation in progress (locating); reverse-geocoding in progress (reverseLoading); saving address (saving); within delivery radius / outside delivery radius; search results dropdown open; editing existing address inline. Live Updates: yes — distance label, within-radius chip, and reverse-geocoded label all update live as the pin moves.

2.15 Customer-Side Shared Components

TopBar (Mobile Portrait)

- Mobile-only (hidden in landscape via `landscape:hidden`). Brand-softer → background gradient strip.
- Location chip button — MapPin icon in white circle, 'DELIVER TO · 25-35 min' label, current address label (or 'Select delivery location'), ChevronDown icon → navigates to location view.
- Address label logic: finds the address matching `selectedAddressId`, else the default, else 'Select delivery location'; truncated to 32 chars; format: ', '.
- Notifications bell button (white circle) → navigates to orders view; small brand dot indicator in the top-right of the bell.
- Profile button — navigates to profile (if logged in) or `goToLogin('profile')` (if not).
- Fetches `/api/addresses` via react-query (enabled only when authed).

TopNav (Landscape)

- Sticky top, hidden in portrait via `hidden landscape:block, backdrop-blur`.
- Brand mark — BrandIcon (36px) + BrandWordmark (26px), click → home.
- Delivery location chip — same logic as mobile TopBar (label, ChevronDown, click → location).
- Centered nav tabs (Home / Menu / Cart) with active-tab highlight (brand-softer bg, brand text, thicker icon stroke) and cart badge (count shown if count > 0).
- Active-tab mapping: `cart/checkout/location` → cart; `categories/category-listing/item-detail` → categories; everything else → home.
- Profile button (right side) — shows `profile.name` (or email username) if authed, else 'Sign in'; navigates to profile or login accordingly.

BottomNav (Portrait)

- Sticky bottom, hidden in landscape via `landscape:hidden`. 3-column grid, max-width md.
- Each tab: icon (24px) with active-state brand color + thicker stroke (2.5 vs 2) + label (11px medium).
- Cart tab badge — absolute-positioned brand-bg pill showing count when count > 0.
- Same active-tab mapping as TopNav. `suppressHydrationWarning` on each tab button (cart count may differ between SSR and client due to `localStorage`).
- `pb-[env(safe-area-inset-bottom)]` for iPhone notch support.

StickyCartBar

- AnimatePresence-mounted only when count > 0. Spring animation in/out (y: 60 → 0, stiffness 300, damping 28).
- Fixed position, bottom-[calc(4rem+env(safe-area-inset-bottom))] (clears the bottom nav), z-40 (above nav's z-30).
- Brand-bg pill with ShoppingBag icon (with animated count badge that scales in on change), 'N item(s) added' text, and ' · View Cart →' CTA.
- Clicking anywhere on the bar navigates to cart view. Hidden in landscape (the TopNav cart tab is used instead).

ItemCard

- Motion.div with opacity+y enter animation (delay index × 0.04 capped at 0.3s).
- Image area: aspect-square, shimmer placeholder (ShimmerImage) until image loads, then group-hover scale-110 zoom. Or UtensilsCrossed icon fallback if no imageUrl.
- Veg indicator (top-left, emerald-600 square border + dot). HeartButton (top-right, white/90 backdrop) — toggles favorite; gating: requires logged-in profile (else toast 'Please sign in to save favorites' + goToLogin('item-detail')). Animated heart scale on toggle.
- Rating badge (bottom-left, black/75 backdrop, amber star) if item.rating > 0.
- Title (line-clamp-1, semibold), description (line-clamp-2, muted).
- Price (extrabold) + ADD button (brand pill, hover brightness, active scale-95) → addItem(); OR QtyStepper (sm) if qty > 0.
- Click on image/title opens openItem(slug).

ItemRow

- Motion.div with opacity+x enter animation (delay index × 0.03 capped at 0.25s). 80×80 image with ShimmerImage (or fallback icon).
- Veg indicator (smaller). Title (line-clamp-1), description (line-clamp-1), price (extrabold).
- Click on image/text opens openItem(slug). ADD button (brand pill) or QtyStepper (sm) on the right.

QtyStepper

- Three sizes: sm (h-7, 5×5 buttons), md (h-9, 7×7 buttons), lg (h-12, 9×9 buttons).
- Pill container with brand border + brand-softer bg + brand text.
- Minus button (white circle, brand text, hover:bg-brand/10). Centered qty display (tabular-nums, foreground color, min-w-4). Plus button (brand bg, brand-foreground text, hover:opacity-90).
- e.stopPropagation() on both buttons (so the click doesn't bubble to parent card openers). Aria-labels 'Decrease quantity' / 'Increase quantity'. Inline SVG Plus/Minus icons.

ImageGallery

- Empty images → UtensilsCrossed placeholder block. Single image → tap to open Lightbox; Expand icon overlay.
- Multi-image carousel: AnimatePresence slide animation (x: 30 → 0 → -30). Prev/Next chevron buttons (only when not at start/end). Page indicator 'N / M' pill at bottom-center. Expand (fullscreen) button at bottom-right. Dot indicators (active = wider pill, inactive = small dot).
- Lightbox modal: Full-screen black/90 backdrop. Close button (top-right), Prev/Next chevrons. Image container h-[80vh] w-full max-w-2xl, object-contain. Click backdrop closes; click image does not. Page indicator 'N / M'. Scale-in animation on active image change.

OrderRating

- Pre-submit state: 'Rate your order' heading, 5 interactive star buttons (hover state highlights), optional review textarea, 'Submit Rating' button (disabled if rating < 1 or submitting).
- Validation: toast 'Please select a star rating' if rating is 0 on submit. Submit POST /api/orders//rate with {rating, review?}.

- Post-submit state: emerald card with CheckCircle2 icon, 'Rated N stars', 'Thanks for your feedback!', 5 display stars (filled up to submittedRating).
- Submit-button spinner via Loader2 animate-spin while submitting. Animations on hover (stars fill amber), active (scale 0.9), post-submit (scale-in motion).

Other shared: BrandWordmark, BrandIcon, Providers, AuthProvider, InstallPrompt, ServiceWorkerRegister — detailed under § 7 Cross-Cutting Systems.

3. Admin-Side PWA

The admin PWA is mounted at /admin as a separate Next.js route segment with its own metadata, manifest, service worker, install prompt, and offline fallback page. Authentication uses a hardcoded email/password checked against environment variables, with a single httpOnly cookie (ab_admin === '1') valid for 12 hours. The dashboard is a single page with five tabs (Orders, Menu, Featured, Analytics, Reviews) and four modal editors (OrderDetailModal, ItemEditor, CategoryEditor, plus a generic Modal helper).

3.1 Admin Route Page

File: src/app/admin/page.tsx

- Wraps the entire admin route in AdminAuthProvider (React Context).
- Renders AdminBackGuard — injects a sentinel history.pushState so the Android/iOS hardware back button cannot immediately leave the website from /admin; first back-press navigates to / (public Home), Home is the final exit boundary.
- Renders AdminGate which conditionally shows one of three views based on auth state.
- Sets up popstate listener that routes user to / on back-button. Cleans up the popstate listener on unmount.
- UI States: loading → centered Loader2 spinner on bg-muted/30 (auth check in flight); !authed → renders ; authed → renders .

3.2 Admin Layout

File: src/app/admin/layout.tsx

- Sets admin-specific metadata.title template '%s · Apna Baithak Admin' (default 'Apna Baithak Admin').
- Sets admin description: 'Admin dashboard for Apna Baithak — manage orders, menu items, reviews and UPI verifications.'
- Mounts admin manifest /admin-manifest.json (separate PWA identity from customer).
- Sets applicationName to 'Apna Baithak Admin'. Enables appleWebApp.capable = true, statusBarStyle = 'black-translucent', title 'Apna Baithak Admin'.
- Declares 5 admin icons (16, 32, 48, 192, 512) all sourced from /admin-brand/*. Sets admin shortcut icon to /admin-brand/favicon.ico. Sets admin Apple touch icon to /admin-brand/apple-touch-icon.png.
- Sets viewport.themeColor to #1A1A1A (dark — blends with admin dark chrome on iOS status bar). Locks viewport to width: device-width, initialScale: 1, maximumScale: 1, userScalable: false (prevents pinch-zoom).
- Mounts (admin SW) and (admin install banner).

3.3 AdminAuthProvider

File: src/components/admin/admin-auth.tsx

- Creates a Context exposing { authed, login, logout, loading }.
- On mount, pings GET /api/admin/stats to verify the existing cookie is still valid (any 2xx → authed).
- login(email, password) → POSTs to /api/admin/login, returns { error } or { error: null }.
- logout() → clears the ab_admin cookie by writing max-age=0 to document.cookie, sets authed=false, calls router.refresh().
- Exports useAdminAuth() hook with a guard that throws if used outside the provider.
- Treats fetch failures as authed=false (network error → not authenticated).

- UI States: loading=true initially, false once the verify-ping resolves; authed=true if stats endpoint returns ok; authed=false if stats endpoint returns non-ok or rejects.

3.4 AdminLogin

File: `src/components/admin/admin-login.tsx`

- Renders brand icon (BrandIcon size 64) and brand wordmark (BrandWordmark height 32) at top of card.
- Subtitle 'Restaurant owner access only'.
- Email input (type=email, required, placeholder owner@example.com, Mail icon prefix, focus ring).
- Password input (type=password, required, placeholder dots, Lock icon prefix, focus ring).
- Submit button labelled 'Sign in' / spinner ('Loader2 animate-spin') while busy.
- On submit: prevents default, sets busy=true, calls login(email, password).
- Success → toast.success('Welcome, owner'). Failure → toast.error(message from API or 'Invalid credentials').
- Disabled submit button while busy. HTML5 required validation prevents empty submit. type=email provides browser-level format validation.

3.5 AdminDashboard (Primary Hub)

File: `src/components/admin/admin-dashboard.tsx` (1599 lines)

Purpose: Top-level admin dashboard shell — sticky header, stats grid, horizontal tab navigation, and the bodies of Orders, Menu, Featured tabs; renders child panels (SalesAnalytics, ReviewsPanel, FeaturedTab) for the other tabs.

- Sticky top header with brand icon, wordmark, 'Restaurant owner dashboard' subtitle.
- Order Acceptance Toggle button in header — green 'Accepting Orders' / red 'Orders Paused' (mobile shows 'Open'/'Paused'); immediately PATCHes `/api/admin/settings` with `{ isAcceptingOrders }`; pulses while toggling; invalidates settings cache; toasts success.
- Logout button in header (calls `logout()` from auth context).
- 4-card stats grid (responsive 2-col mobile / 4-col desktop): Today's Orders count; Today's Revenue (📦 formatted); Active Orders count; Total Revenue (📦 formatted).
- StatCardSkeleton shown while stats loading.
- Horizontally scrollable tab bar (no-scrollbar utility) with 5 tabs: Orders, Menu, Featured, Analytics, Reviews. Each tab has an icon (LayoutDashboard, UtensilsCrossed, Star, TrendingUp, Star respectively) and label.
- Live new-order polling every 15s: fetches `/api/admin/orders`, compares to known IDs in a `useRef`, toasts 'New order {orderNumber} — 📦X' (with Bell icon, 6s duration) for each new one, invalidates both `admin-orders` and `admin-stats` queries when new orders arrive.
- Seeds the known-IDs set on first load so existing orders don't trigger toasts.
- `updateStatus(id, status) → PATCH /api/admin/orders/:id` with `{ status }`, toasts success, invalidates `orders+stats`.
- `updatePayment(id, paymentReceived, method?) → PATCH /api/admin/orders/:id` with `{ paymentReceived, paymentReceivedMethod }`, toasts success/failure, invalidates `orders+stats`, returns boolean.
- `upiReview(id, action: 'approve'|'reject', note?) → PATCH /api/admin/orders/:id/upi-review` with `{ action, note }`, toasts success ('Payment approved — order marked as Paid, Telegram sent' / 'Payment rejected — flagged for follow-up'), invalidates `orders+stats`, returns boolean.

3.6 Orders Tab (sub-component)

- Status filter pills row (horizontally scrollable): All, New, Preparing, Out for Delivery, Delivered, Cancelled — selected pill gets bg-brand text-brand-foreground.
- '⚠ Pending UPI Verification' toggle pill (amber when active) — sends `upiStatus=PENDING_VERIFICATION` query param.
- 'Select' button (toggles bulk mode) — only visible when `orders.length > 0`.
- Bulk action bar (when bulk mode AND `selectedIds.size > 0`) with: '{n} selected' label; one 'Mark {status}' button per non-cancelled status (NEW, PREPARING, OUT_FOR_DELIVERY, DELIVERED); 'Cancel Orders' button (red outline); 'Clear' link.
- Select all / Deselect all bar (when bulk mode and orders present).
- Per-order checkbox selection toggle in OrderCard.
- `bulkUpdateStatus(status)` fires parallel PATCHes for all selected IDs, toasts result, clears selection, exits bulk mode, invalidates caches.
- Order cards list (animated with framer-motion stagger).
- OrderDetailModal rendered conditionally when an order is selected.
- Local mutation of selected order after status/payment/UPI-review actions (so modal reflects changes immediately without refetch).

OrderCard sub-component features:

- Animated entrance (framer-motion fade-up). Status-colored circular icon (emerald/red/purple/amber/brand based on status).
- Order number, status badge, payment status badge ('Paid (Cash)' / 'Paid (UPI)' / 'Payment Pending').
- Customer name · total items count · relative time. Right-aligned total amount + payment mode label.
- ChevronRight indicator (non-bulk mode). Selection circle (bulk mode).
- Quick payment action footer (non-bulk mode only): 'Cash' button (emerald when active); 'UPI' button (emerald when active); 'Mark Pending' button (orange) shown only when payment is already received.
- `stopPropagation` on quick action buttons so they don't open the modal. Bulk-selected visual state (border-brand + ring).

3.7 Menu Tab (sub-component)

Sub-tab toggle: Items | Categories

Items sub-tab features:

- Items header with item count ('Menu Items (N)') and 'Add item' button.
- Item list with thumbnail, name, veg indicator (green dot), HIDDEN badge if unavailable, '{category} · ₹price · {N}/5 images'.
- Edit (Pencil) button per item → opens ItemEditor. Delete (Trash2) button per item → `confirm()` then DELETE call, toast.
- Empty state when no items. Loading skeleton ×6. ItemEditor rendered when editing or creating is set.

Categories sub-tab (CategoriesManager) features:

- Header with count 'Categories (N)' and 'Add category' button.
- Sortable category list (sorted by `sortOrder`): Up/Down arrow buttons to swap `sortOrder` with neighbor (two parallel PATCH calls); Disabled up arrow on first item, disabled down on last.
- Category icon (image if set, else UtensilsCrossed placeholder). Category name, HIDDEN badge if `isActive`, 'Order: N · M items'.
- Edit (Pencil) button → opens CategoryEditor. Delete (Trash2) button → blocked with toast if items still assigned; otherwise `confirm()` then DELETE.

- Loading pulse skeletons ×4. Empty dashed-border state. CategoryEditor modal when editing/creating.

CategoryEditor (modal) features:

- 'Add category' / 'Edit category' title. 1:1 icon preview + 'Upload icon' button (image/* file input, POSTs to /api/admin/menu/categories/:id/icon as FormData) — disabled until saved.
- Name input (required). Description textarea (optional). Display order numeric input (integer only). Active checkbox ('visible to customers').
- Cancel / Save buttons (Save disabled while saving). Creates via POST /api/admin/menu/categories or updates via PATCH /api/admin/menu/categories/:id.
- Toasts success/failure. Creating (Save label: 'Create category'); Editing (Save label: 'Save changes'); Saving (label: 'Saving...', disabled); Uploading icon (spinner); Icon-not-yet-saved (text: 'Save the category first, then upload an icon.').

ItemEditor (modal) features:

- 'Add menu item' / 'Edit menu item' title. Name input (required). Price input (₹, numeric-only, required).
- Category select dropdown (required; defaults to first category). Description textarea. Prep time (mins) numeric input. Calories numeric input. Vegetarian checkbox. Available checkbox.
- Images section: Counter 'Images (N/5)'; 'Upload' button (hidden when at MAX_ITEM_IMAGES = 5); 4-col grid of square image thumbnails with remove (X) button; Empty state message ('Save the item first, then upload images.' when creating).
- Cancel / Save buttons (Save disabled while saving). Creates via POST /api/admin/menu/items or updates via PATCH /api/admin/menu/items/:id.
- After creating, calls window.location.reload() to switch the editor into edit mode for the new item. Toasts success/failure.
- For edit mode: fetches existing item data via useQuery(['admin-item', itemId]) then populates form via useEffect. Upload image: POST FormData to /api/admin/menu/items/:id/images; adds to local images array.

3.8 OrderDetailModal

File: src/components/admin/order-detail-modal.tsx (376 lines)

Purpose: Bottom-sheet-on-mobile / centered-modal-on-desktop showing full order detail with timeline, items, bill breakdown, payment marking controls, UPI review section, receipt printing, and delete.

- Animated entrance (framer-motion fade + slide up on mobile, fade on desktop). Backdrop click closes; inner stops propagation.
- Header: order number, status badge, formatted date-time, close (X) button.
- Customer card: avatar icon, customer name, phone, 'Call' (tel:) and 'SMS' (sms:) action buttons (only shown if phone exists).
- Delivery address block: address line, distance in km.
- Exact Location panel (when lat/lng present): Lat/lng readout (mono font); 'View on map' link → Google Maps search; 'Get directions' link → Google Maps directions from restaurant to customer (driving mode).
- Status Timeline (NOT shown for CANCELLED): 4-step horizontal timeline (NEW → PREPARING → OUT_FOR_DELIVERY → DELIVERED) with icons (Package, ChefHat, Bike, HomeIcon), done/active/upcoming states, animated pulsing ring on active step, description text below.
- Cancelled banner (red, XCircle icon, 'This order was cancelled.')
- Items list: per-item image, name, 'qty × ₹price', line total.
- Bill Details: Item Total, Delivery Fee (or 'FREE'), Distance (admin reference), Total (large), checkout payment mode + payment status footer.

- Payment Received panel: Current state badge ('Paid (Cash)' / 'Paid (UPI)' / 'Pending'); 'Received as Cash' button; 'Received as UPI' button; 'Reset to Pending' button (only when paymentReceived is true); Explanatory text about door payment vs. checkout choice.
- Customer Note card (amber) — only if order.notes present.
- UPI Payment Verification section (when paymentMode === 'UPI' and upiStatus !== 'NONE') — see below.
- Footer actions: 'Print Receipt' button → fetches PDF from /api/orders/:id/receipt?admin=1 as blob, creates object URL, triggers <a download>.click() (named receipt-{orderNumber}.pdf), revokes URL after 60s.
- 'Delete Order' button (red outline) → window.confirm() with detailed warning, DELETE /api/admin/orders/:id, closes modal, window.location.reload().
- 'Advance to next status →' button (hidden for DELIVERED/CANCELLED) — auto-advances NEW→PREPARING→OUT_FOR_DELIVERY→DELIVERED.
- Status override buttons: one per non-current status (NEW, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED) — CANCELLED styled red.

UpiVerificationSection (sub-component) features:

- Header: 'UPI Payment Verification' + colored status pill.
- Verified UTR display (emerald, mono code) — only if present. Uploaded screenshot thumbnail (image link opens in new tab) — only if upiScreenshotUrl present.
- AI Verification Result panel (when attempt status not AWAITING_SCREENSHOT/EXPIRED): Authentic: Yes/No/—; Confidence: percentage; Extracted Amount: ₹X; Extracted Payee; Extracted Timestamp; Extracted UTR; AI reasoning text (italic); Failed checks list (amber): screenshot authenticity, ±10min timestamp, amount match, payee match, UTR, gemini_api_error.
- Admin review UI (only when upiStatus === 'PENDING_VERIFICATION'): Optional admin note text input; 'Approve (mark Paid)' button (emerald); 'Reject (flag)' button (red outline); Explanatory text: 'Approving marks the order as Paid (UPI) and sends the Telegram payment-received alert.'
- Attempt metadata footer: 'Attempt #N · Ref: ... · Payee: ...'
- UI States: Awaiting screenshot (no review UI, no AI panel); Pending verification (review UI active); Paid (no review UI, verified banner); Expired (no review UI, red status); Busy state during review action (buttons disabled).

3.9 FeaturedTab

File: src/components/admin/featured-tab.tsx (349 lines)

Purpose: Two-section panel: drag-to-reorder featured items list + star-toggle for all menu items.

- Section 1 — Featured Items (drag to reorder): Header with star icon, 'Featured Items' title, count badge. 'Saving...' indicator while reorder is in flight (Loader2 + text). Hint text: 'Drag items up/down to set the display order on the homepage.' Empty state when 0 featured: dashed border + Star icon + 'No featured items yet' + helper text.
- Sortable rows (when populated): DndContext + SortableContext with verticalListSortingStrategy. Sortable rows each contain: Drag handle (GripVertical, touch-none, cursor-grab / active:cursor-grabbing); Position number badge (1-indexed, brand-soft); Thumbnail (image or ImageIcon placeholder); Name + price; Remove (X) button (red hover). While dragging: border-brand, elevated shadow, opacity 0.8, z-index 10.
- Section 2 — All Menu Items (toggle featured): Header with Plus icon, 'All Menu Items' title, count badge. Hint: 'Tap the star to add/remove items from the Featured section above.' Loading state: centered spinner.
- List of all items with thumbnail, name, '{price} · {category}', and star toggle button: Amber-filled star when featured (bg-amber-100 text-amber-600); Muted star when not featured (bg-muted text-muted-foreground).

- toggleFeatured(itemId, currentlyFeatured) → PATCH /api/admin/menu/featured with { itemId, isFeatured: !currentlyFeatured }; invalidates admin-featured, admin-menu-items, AND menu-featured (public homepage cache).
- onDragEnd handler: Computes old/new indices; arrayMove to produce new order locally; Optimistically sets query data (qc.setQueryData) for instant UI feedback; PATCHes the full orderedIds array to /api/admin/menu/featured; On success: invalidates admin-featured + menu-featured; On failure: toast.error + reverts via invalidate (server truth); Sets reordering=true during the request.
- Sensors configured: PointerSensor with activationConstraint: { distance: 5 } (prevents accidental drag on click of remove button); KeyboardSensor with sortableKeyboardCoordinates (accessibility).
- Collision detection: closestCenter. Axis: vertical (verticalListSortingStrategy).

3.10 ReviewsPanel

File: src/components/admin/reviews-panel.tsx (144 lines)

Purpose: Read-only customer reviews & ratings dashboard with summary stats and review list.

- 4-card summary grid (responsive 2-col mobile / 4-col desktop): Avg Rating card (Star icon, amber accent, '—' if 0 reviews); Total Reviews card (MessageSquare icon, blue accent); Rating Distribution card (col-span-2): 5 horizontal bars (5★ to 1★, descending). Each bar: star count label, animated fill width (framer-motion from 0% to {pct}% with staggered 0.1s delay), count number. Bars use amber-400 fill.
- Reviews list: Header 'Customer Reviews'. Loading state: centered spinner. Empty state: MessageSquare icon, 'No reviews yet.', 'Customer ratings will appear here once orders are delivered and rated.'
- Each review card (animated entrance, staggered 0.04s): Customer name + order number; Subtitle: relative time · $\frac{\text{total}}{\text{items}}$ string; 5-star display (filled amber for $n \leq \text{rating}$, muted otherwise); Review text in quotes (only if r.review present).
- Read-only — no moderation features.

3.11 SalesAnalytics

File: src/components/admin/sales-analytics.tsx (201 lines)

Purpose: 7-day revenue bar chart, top-selling items list, and CSV export buttons.

- Export buttons row: 'Export Orders (CSV)' — fetches /api/admin/export?type=orders as blob, downloads with Content-Disposition filename or fallback apna-baithak-orders.csv; toasts success/failure. 'Export Revenue (30-day CSV)' — same flow with type=revenue.
- 7-day revenue chart card: Header with TrendingUp icon + 'Last 7 Days Revenue'. Subtitle: ' $\frac{\text{total7d}}{\text{total}}$ total · {n} orders'. Today revenue callout (right-aligned).
- Recharts BarChart: XAxis (date label, fontSize 11); YAxis ($\frac{\text{total}}{\text{orders}}$ formatter, $\frac{\text{total}}{\text{orders}}$ 1k+ shows '1k', fontSize 10, width 40); Tooltip with custom content style (white bg, rounded-12, shadow), formatter shows $\frac{\text{total}}{\text{orders}}$ for revenue / '{n} orders' for orders; Bar with rounded top corners (radius 6,6,0,0); Each Cell colored — last day (today) is #FF5252 (brand red), other days #FFB3B3 (light pink); Cursor fill: rgba(255, 82, 82, 0.08).
- Loading state: 192px-tall spinner. Responsive container at 100% width, 192px height.
- Top Selling Items card (last 30 days): Header: Package icon + 'Top Selling Items' + '(last 30 days)' subtitle. Empty state: Package icon + 'No sales data yet'. Each item row: rank number (#1, #2, ...), thumbnail, name, $\frac{\text{revenue}}{\text{qty}}$ revenue, qty-progress-bar (bg-brand), '{qty} sold' label. Bar width relative to top item's qty.
- useQuery(['admin-sales'], ...) with refetchInterval: 60000 (auto-refresh every 60s).

3.12 Admin PWA Components

AdminInstallPrompt + AdminServiceWorkerRegister + offline.html — detailed under § 7 Cross-Cutting Systems.

4. Backend & API Inventory

The backend consists of 35 Next.js Route Handler files under `src/app/api/`. They expose 46 distinct method+path combinations (counting each verb on the same path as a separate endpoint).

Customer-facing routes are mostly public or protected by Supabase JWT; admin routes are guarded by a single boolean cookie check (`ab_admin === '1'`). The Telegram webhook at `/api/telegram/webhook` is a separate surface that accepts `callback_query` events from inline keyboard buttons.

4.1 Customer-Facing Endpoints

Method · Path	Auth	What it does
GET /api/menu/items	Public	List available menu items, optionally filtered by ?category= or ?search=. Hard filter isAvailable:true. Sorted by sortOrder asc, name asc. Returns first image per item.
GET /api/menu/items/[slug]	Public	Fetch a single menu item by slug with ALL images and category. Returns the item regardless of isAvailable (so customers can view linked/disabled items from history). Images sorted by sortOrder asc.
GET /api/menu/categories	Public	List all active categories with itemCount (computed via _count filtered to isAvailable:true items). Sort by sortOrder asc.
GET /api/menu/featured	Public	Return featured menu items for the homepage rail, ordered by featuredOrder asc, name asc. Filter isFeatured:true AND isAvailable:true. Returns only first image per item.
POST /api/checkout	Supabase JWT	Create a new order from cart. Steps: check isAcceptingOrders (503 ORDERS_PAUSED if paused); validate phone (normalize Indian mobile); validate items/addressId; verify address ownership; check distance serviceable (10km); server-side price re-validation; computeDeliveryCharge; generate order number AB-YYYY-NNNN via MAX+1 (5 retries for P2002); Order.create + OrderItem[] + OrderStatusHistory(NEW); fire notifyNewOrder Telegram non-blocking.
GET /api/orders	Supabase JWT (graceful)	List the current customer's order history (last 50, newest first). Includes items and statusHistory. Returns empty array if not signed in (not 401).
POST /api/orders/[id]/rate	Supabase JWT	Submit a 1-5 star rating + optional review (trimmed, sliced to 500 chars). Validates rating between 1-5. Order must exist, belong to caller, status===DELIVERED, rating not already set. Returns {ok:true, rating, review}.
GET /api/orders/[id]/receipt	PUBLIC (un guessable CUID)	Download a PDF receipt for an order. Calls buildReceiptPdf(receiptOrder) from src/lib/receipt-pdf.ts. Returns binary PDF with Content-Disposition attachment; filename=receipt-.pdf. ?admin=1 query param accepted but ignored.
POST /api/orders/[id]/upi/initiate	Supabase JWT	Start a UPI payment attempt. Expires all prior AWAITING_SCREENSHOT attempts (EXPIRED). Creates PaymentAttempt (status:AWAITING_SCREENSHOT, expiresAt=now+5min). Updates Order.upiStatus=AWAITING_SCREENSHOT. Returns {attempt, deepLink, qrDataUrl}.
POST /api/orders/[id]/upi/upload	Supabase JWT	Upload UPI screenshot + Gemini Vision verify + auto-mark PAID if all checks pass. multipart/form-data with 'screenshot' (image, ≤5MB) and 'attemptId'. Side effects: upload to Supabase Storage media bucket; Gemini API call (gemini-2.5-flash); 4 verification checks (authenticity, timestamp ±10min, exact amount, payee match); on success Order.upiStatus=PAID + notifyPaymentReceived Telegram; on failure Order.upiStatus=PENDING_VERIFICATION + notifyPaymentPendingManual Telegram with inline Verify/Reject buttons + store telegramMessageIds for multi-device sync.
GET /api/orders/[id]/upi/status	Supabase JWT	Poll current UPI payment state. AUTO-EXPIRES latest attempt if now > expiresAt (lazy expiry). Returns {order, attempt}.
POST /api/orders/[id]/upi/manual	Supabase JWT	Customer clicks 'Continue without screenshot' — request manual admin verification. Marks latest attempt REJECTED with failedChecks='customer_manual_review'. Updates Order.upiStatus=PENDING_VERIFICATION. Fires notifyPaymentPendingManual Telegram.

Method · Path	Auth	What it does
GET /api/addresses	Supabase JWT (graceful)	List customer's saved addresses (sorted by isDefault desc, createdAt desc). Returns empty list silently for unauthenticated callers.
POST /api/addresses	Supabase JWT	Create new saved address. Auto-defaults if first address OR isDefault=true (bulk clears existing defaults). Required fields: houseFlat, streetArea, city, pincode, latitude, longitude.
PATCH /api/addresses/[id]	Supabase JWT	Update an address (partial update via ?? fallback). If isDefault=true, clears all other addresses' defaults first. Ownership check.
DELETE /api/addresses/[id]	Supabase JWT	Delete a saved address. Ownership check. Does NOT reassign default to another address if the deleted one was default.
GET /api/settings	Public	Public endpoint exposing ONLY isAcceptingOrders boolean from RestaurantConfig. Defaults to true if row missing or DB unreachable (fail-open).
GET /api/auth/me	Supabase JWT	Get current customer's profile, auto-creating if missing (Supabase user ↔ Customer row sync). Email-based fallback merge handles Supabase ID changes.
PATCH /api/auth/me	Supabase JWT	Update customer's name and/or phone. Calls Supabase Admin API to sync user_metadata.full_name/phone. Validates name (≤100 chars), phone (Indian mobile normalization, stored as +91).
GET /api/ratings	Public	Public aggregate rating for the restaurant (avg + count of all delivered-order ratings, 1 decimal place).
POST /api/telegram/webhook	Telegram secret	Receive Telegram Bot webhook events. Handles callback_query events (Verify/Reject button presses). Verifies X-Telegram-Bot-API-Secret-Token header against TELEGRAM_WEBHOOK_SECRET env var. Fail-closed: if env var missing, 503 for ALL requests.
GET /api/telegram/webhook	Public	Simple health-check endpoint.

4.2 Admin Endpoints

All admin endpoints (except /api/admin/login) use isAdminAuthorized(req) — checks that the ab_admin cookie equals the literal string '1'. Cookie is httpOnly, sameSite=strict, maxAge=12h, path=/.

Method · Path	Auth	What it does
POST /api/admin/login	Public	Hardcoded admin credential check. Email is toLowerCase().trim()-ed, password compared as-is. On match: returns {ok:true} and sets Set-Cookie: ab_admin=1 (httpOnly, sameSite=strict, maxAge=43200s/12h, path=/).
GET /api/admin/orders	Admin	List all orders (newest first, take:200), filterable by ?status, ?upiStatus, ?from=YYYY-MM-DD, ?to=YYYY-MM-DD. Adds computed fields: paymentReceived normalized to boolean, latitude/longitude pulled from linked address, latestAttempt.
PATCH /api/admin/orders/[id]	Admin	Update order status and/or admin payment-received tracking. If status provided: appends OrderStatusHistory entry. Legacy auto-mark: if status===DELIVERED AND paymentStatus===PENDING, the OLD paymentStatus field is auto-set to PAID. paymentReceivedMethod must be Cash or UPI.
DELETE /api/admin/orders/[id]	Admin	Permanently delete an order (cascades to items, history, payment attempts via Prisma schema). Verify exists first (404 if not).

Method · Path	Auth	What it does
PATCH /api/admin/orders/[id]/upi-review	Admin	Admin manually approves or rejects a PENDING_VERIFICATION UPI order (web UI equivalent of Telegram Verify/Reject). Body {action: 'approve' 'reject', note?}. Approve: Order.upiStatus=PAID + PaymentAttempt.status=VERIFIED + notifyPaymentReceived Telegram. Reject: Order stays PENDING_VERIFICATION; PaymentAttempt.adminNote set. NOTE: web UI route does NOT edit Telegram messages (asymmetric with webhook).
GET /api/admin/settings	Admin	Get full restaurant config (currently just isAcceptingOrders). Auto-creates singleton row (id=1) if missing.
PATCH /api/admin/settings	Admin	Toggle isAcceptingOrders (the ONLY way to pause/resume orders). RestaurantConfig.upsert (id=1). Change takes effect immediately on next /api/checkout call.
GET /api/admin/export?type=orders revenue	Admin	Download CSV. Orders export: last 1000 orders, full customer info, items joined as '2x Paneer; 1x Naan', payment info, distance, address, notes. Revenue export: last 30 days, one row per day, {date, orders, items, revenue}. 30 sequential DB queries.
GET /api/admin/reviews	Admin	List all customer ratings + reviews with aggregate stats (avgRating 2 decimals, totalReviews, ratingCounts[5] array, reviews list with order info, items string).
GET /api/admin/menu/items	Admin	List ALL menu items (including unavailable + featured + featuredOrder) for admin management. Sorted sortOrder asc, name asc. Includes category + images sorted by sortOrder asc.
POST /api/admin/menu/items	Admin	Create a new menu item, optionally with initial images (capped at MAX_ITEM_IMAGES=5). Auto-generates slug from name + timestamp if not provided. Image URLs/dataURLs stored as-is.
GET /api/admin/menu/items/[id]	Admin	Get a single menu item with all images + category (admin view).
PATCH /api/admin/menu/items/[id]	Admin	Update item fields; optionally REPLACE the entire image set if body.images is provided (destructive: deleteMany + create). Old image files in Supabase Storage NOT deleted (orphaned files accumulate).
DELETE /api/admin/menu/items/[id]	Admin	Delete menu item + its Supabase Storage images. Fetches all MenuItemImage rows, parses URLs to extract storage path, calls supabase.storage.from('menu-items').remove(paths). MenuItem.delete cascades to image rows; order_items.itemId set null per schema, preserving order history.
POST /api/admin/menu/items/[id]/images	Admin	Upload a single image to an item. multipart/form-data with field 'file' (image, ≤5MB). Validates count < 5. Uploads to Supabase Storage menu-items bucket at path {slug-or-id}/{timestamp}-{rand6}.{ext}. Filename uses slug-or-id + timestamp + 6 random base36 chars.
GET /api/admin/menu/featured	Admin	List all featured items (for the admin drag-and-drop reorder UI). Filter isFeatured:true, order featuredOrder asc, name asc. Does NOT filter by isAvailable.
PATCH /api/admin/menu/featured	Admin	Toggle an item's featured status OR reorder the entire featured list. TWO MODES: Reorder mode {orderedIds: string[]} sets featuredOrder 0,1,2,... atomically via transaction. Toggle mode {itemId, isFeatured: boolean} — Toggle on: append to end (max featuredOrder + 1). Toggle off: clear featuredOrder to null AND re-sequence remaining items to close gaps.
GET /api/admin/menu/categories	Admin	List ALL categories (including inactive) with item counts. Sorted sortOrder asc. No filter.
POST /api/admin/menu/categories	Admin	Create a new category. Name required; slug auto-generated if not provided.

Method · Path	Auth	What it does
PATCH /api/admin/menu/categories/[id]	Admin	Update a category's fields (partial update). If slug provided, slugified.
DELETE /api/admin/menu/categories/[id]	Admin	Delete a category — REFUSES if any menu items are still assigned (409 with detailed message naming count).
POST /api/admin/menu/categories/[id]/icon	Admin	Upload a category icon (square image) to Supabase Storage. multipart/form-data with field 'file' (image, ≤5MB). Uploads to menu-items bucket (SAME bucket as item images) at path categories/{slug-or-id}/{timestamp}-{rand6}.{ext}. Does NOT delete previous icon from storage.
GET /api/admin/stats	Admin	Today's orders/revenue + total counts + menu/category counts (dashboard cards). 4 parallel DB queries via Promise.all. Revenue excludes CANCELLED orders. activeOrders = status in {NEW, PREPARING, OUT_FOR_DELIVERY}.
GET /api/admin/stats/sales	Admin	Last 7 days revenue + order count per day + top 5 selling items (last 30 days). 7 sequential DB queries (one per day). Day label: weekday short name via toLocaleDateString('en-IN', { weekday: 'short' }).

4.3 Telegram Webhook Details

Endpoint: POST /api/telegram/webhook. Commands handled:

- verify:<orderId> — admin tapped '☑ Verify Payment' inline button. Marks order PAID, marks latest attempt VERIFIED with adminReviewedBy='telegram', fires notifyPaymentReceived, edits ALL stored Telegram messages to a '☑ Verified' template, answers the callback query with a toast.
- reject:<orderId> — admin tapped '☑ Reject' inline button. Marks latest attempt with adminReviewedBy='telegram', adminNote='Rejected via Telegram button'. Order stays PENDING_VERIFICATION. Edits ALL stored Telegram messages to a '☑ Rejected' template. Does NOT fire a new notification.
- Malformed/unknown callback data — answers 'Unknown action' and exits.
- Non-callback updates (regular text messages) — ignored, returns 200.

Inline keyboards (built by pendingKeyboard):

- Row 1: '☑ Verify Payment' (callback_data: verify:{orderId}) | '☑ Reject' (callback_data: reject:{orderId}).
- Row 2 (only if lat/lng available): '📍 View on Map' (URL → Google Maps search) | '📍 Directions' (URL → Google Maps directions, with restaurant coords as origin).
- Note: Telegram does not allow tel: URLs in inline buttons; the customer phone is shown as text in the message body where Telegram auto-links it.

Multi-chat message sync (the critical feature):

- Bot reads TELEGRAM_CHAT_IDS (comma-separated list) — falls back to legacy singular TELEGRAM_CHAT_ID if plural unset.
- Each sendTelegramMessage fires in PARALLEL to all chats and returns [{chatId, messageId, ok}].
- Caller (POST /api/orders/[id]/upi/upload and /api/orders/[id]/upi/manual) persists the successful {chatId, messageId} pairs as a JSON string in order.telegramMessageIds.
- When admin presses Verify/Reject on ONE device, editAllTelegramMessagesForOrder(orderId, text) parses telegramMessageIds, then calls editMessageText for EACH pair in parallel — so every device's message updates to '☑ Verified' or '☑ Rejected'.

Auth/security:

- Fail-closed: if TELEGRAM_WEBHOOK_SECRET env var is not set → 503 for ALL requests.
- Telegram sends X-Telegram-Bot-API-Secret-Token header on every webhook call. Must match env var exactly. Mismatch → 401.

4.4 UPI Payment State Machine

From State	To State	Trigger
NONE (initial)	AWAITING_SCREENSHOT	POST /api/orders/[id]/upi/initiate (creates PaymentAttempt, starts 5-min timer)
AWAITING_SCREENSHOT	PAID	POST /api/orders/[id]/upi/upload when Gemini checks ALL pass
AWAITING_SCREENSHOT	PENDING_VERIFICATION	POST /api/orders/[id]/upi/upload when Gemini checks fail OR Gemini errors out
AWAITING_SCREENSHOT	PENDING_VERIFICATION	POST /api/orders/[id]/upi/manual (no screenshot, manual review requested)
AWAITING_SCREENSHOT	EXPIRED	GET /api/orders/[id]/upi/status (auto-expire on read if now > expiresAt)
AWAITING_SCREENSHOT	EXPIRED	POST /api/orders/[id]/upi/upload when window already expired (marks attempt EXPIRED, returns 400)
AWAITING_SCREENSHOT	EXPIRED	POST /api/orders/[id]/upi/initiate (bulk-expires prior AWAITING_SCREENSHOT attempts before creating new one)
PENDING_VERIFICATION	PAID	POST /api/admin/orders/[id]/upi-review with action='approve'
PENDING_VERIFICATION	PAID	POST /api/telegram/webhook with verify: callback
PENDING_VERIFICATION	PENDING_VERIFICATION (stays)	POST /api/admin/orders/[id]/upi-review with action='reject' (records adminNote)
PENDING_VERIFICATION	PENDING_VERIFICATION (stays)	POST /api/telegram/webhook with reject: (records adminNote)

Gemini auto-verify criteria (ALL 4 must pass — UTR check removed):

- Authenticity — `gemini.is_authentic === true`. Prompt explicitly tells Gemini that shared receipts (e.g. from GPay's Share button) are VALID; only flag as inauthentic on tampering signs.
- Timestamp — extracted timestamp parses to a valid Date AND $|\text{extracted} - \text{serverTime}| \leq 10 \text{ min}$ (`TIMESTAMP_WINDOW_MS = 10 × 60 × 1000`).
- Amount — `gemini.extracted_amount === order.totalAmount` (exact match, Integer comparison).
- Payee — `matchPayee()` returns true: extracted payee contains expected ID OR name (case-insensitive, trimmed, bidirectional substring match).

4.5 Pricing Engine (in checkout)

Lives in `src/lib/delivery.ts` — imported by both backend (`/api/checkout`) and frontend (`cart/checkout` views) for single-source-of-truth pricing.

Distance & Service Radius:

- `isDistanceServiceable(distKm)` returns `distKm <= RESTAURANT.deliveryRadiusKm` (default 10 km, overridable via `DELIVERY_RADIUS_KM` env).
- Address is pre-checked at selection time AND re-checked defensively in checkout.
- Out-of-radius → 400 'Sorry, we only deliver within 10 km. Your address is X.XX km away.'

Order eligibility (Step 1, distance-first to fix wrong-message-priority bug):

- Far-distance rule (checked FIRST): if `distanceKm > 7` AND `subtotal < 800` → `BELOW_FAR_MIN_SUBTOTAL`. Message: 'Orders beyond 7km require a minimum order of ₹800.'

- Universal rule: if subtotal < 200 → BELOW_MIN_SUBTOTAL. Message: 'Minimum order for delivery is ₹200.'
- Both return remaining (₹ needed to become eligible).

Delivery charge (Step 2):

- If subtotal >= 2000 → finalCharge = 0 (universal free-delivery override, ANY distance).
- Else: raw = 20 + (clamped_distance - 1) × (70 - 20) / (10 - 1) = 20 + 5.5556 × (clamped_distance - 1). Distance clamped to [1, 10]. Rounded to nearest ₹5 via Math.round(raw / 5) × 5.
- So ₹20 at 1 km, ₹70 at 10 km, linear between.

Bill breakdown (final, persisted on Order):

Field	Value
itemTotal	$\Sigma(\text{item.price} \times \text{quantity})$ — re-computed server-side from live menu data
handlingFee	0 (removed; kept in schema for legacy orders)
deliveryFee	computeDeliveryCharge().finalCharge — ₹0 if free override applies, else ₹20-₹70 rounded to ₹5
gstAndCharges	0 (removed; kept in schema for legacy orders)
totalAmount	itemTotal + deliveryFee

Quantity clamping: Each item's quantity is clamped to [1, 50] via `Math.max(1, Math.min(50, Number(i.quantity) || 1))`.

Order number generation:

- Format: AB-YYYY-NNNN where NNNN is zero-padded to 4 digits.
- Sequence derived from `MAX(orderNumber) + 1` for the current year (NOT `COUNT(*) + 1` — survives deletions).
- 5-retry loop handles P2002 (unique constraint) race conditions; each retry bumps the sequence by 1.

Notifications triggered by checkout: `notifyNewOrder(orderId)` — fires on every successful order creation. Sends full order details (customer name, phone as text, address, lat/lng, Google Maps view link, Google Maps directions link, item list, total, payment mode + UPI status, notes) to ALL chats in `TELEGRAM_CHAT_IDS`. Non-blocking — `.catch(console.error)` — checkout never fails on Telegram outage.

4.6 Endpoint Summary Count

Customer-facing API: 22 method+path combinations across 18 files. Admin API: 24 method+path combinations across 17 files. TOTAL: 46 distinct HTTP method+path combinations across 35 route files. (Plus 1 Telegram webhook GET health-check.)

5. Database Schema Inventory

Database: Turso (libSQL / SQLite-compatible) accessed via Prisma 6 with the @prisma/adapters-libsql driver. Schema defined in `prisma/schema.prisma`. 9 tables total — no admin user table (admin login is a hardcoded env-var check), no JWT refresh-token table, no analytics warehouse. The schema comment notes 'Customer account is a mirror of the Supabase Auth user (created on first login).' All money amounts stored as Int (paise-free; values are rupees).

5.1 Category (categories)

Purpose: Top-level menu grouping (e.g. Pizzas, Burgers, Pasta). Each category has an icon image and a sort order; can be hidden by setting `isActive=false`.

- `id` (String, cuid, PK).
- `name` (String, unique).
- `slug` (String, unique) — URL-safe identifier used by `/api/menu/items?category=`.
- `iconUrl` (String, optional) — Supabase Storage URL of the category icon image (uploaded via `/api/admin/menu/categories/[id]/icon`).
- `description` (String, optional) — shown to customers on the category listing page.
- `sortOrder` (Int, default 0) — controls display order; lower comes first.
- `isActive` (Boolean, default true) — when false, category is hidden from public `/api/menu/categories` but still visible to admins.
- `createdAt` (DateTime, default `now()`). `updatedAt` (DateTime, `@updatedAt`).
- `items` (MenuItem[]) — one-to-many relation.
- Indexes: `sortOrder`. `@@map('categories')`.

5.2 MenuItem (menu_items)

Purpose: Individual menu item (e.g. Margherita Pizza, Veg Burger). Up to 5 images per item (enforced in admin upload logic).

- `id` (String, cuid, PK). `name` (String). `slug` (String, unique).
- `description` (String, optional). `price` (Int) — rupees, no decimals.
- `categoryId` (String) — FK to Category; `onDelete: Cascade`.
- `isAvailable` (Boolean, default true) — when false, hidden from public menu but still shown if directly linked from an order.
- `isVeg` (Boolean, default true) — drives the green veg indicator on cards.
- `prepTimeMins` (Int, optional) — drives the prep-time badge on item detail.
- `calories` (Int, optional) — drives the calories badge on item detail.
- `rating` (Float, default 0) — currently a static value set at creation; not derived from order ratings.
- `sortOrder` (Int, default 0) — controls display order within a category.
- `isFeatured` (Boolean, default false) — when true, the item appears in the homepage 'Featured Items' section. Admin toggles this from the menu management screen.
- `featuredOrder` (Int, optional) — the display sequence (0, 1, 2, ...) within the featured section. Admin controls this via drag-and-drop reordering. Null for non-featured items. When an item is unmarked, its `featuredOrder` is cleared and the remaining items are re-sequenced to close the gap.
- `createdAt`, `updatedAt`.
- `images` (MenuItemImage[]) — one-to-many. `orderItems` (OrderItem[]) — one-to-many (with `onDelete: SetNull` so order history is preserved if an item is deleted).
- Indexes: `categoryId`, `isAvailable`, `sortOrder`, `isFeatured`. `@@map('menu_items')`.

5.3 MenuItemImage (menu_item_images)

Purpose: Images attached to a menu item (up to 5 per item). Each image has its own sortOrder so the gallery order is controlled.

- id (String, cuid, PK). itemId (String) — FK to MenuItem; onDelete: Cascade.
- url (String) — Supabase Storage public URL.
- sortOrder (Int, default 0) — gallery order.
- createdAt (DateTime, default now()).
- Index: [itemId, sortOrder] (composite). @@map('menu_item_images').

5.4 Customer (customers)

Purpose: Mirror of the Supabase Auth user (created on first login via /api/auth/me). Stores customer profile info that lives outside Supabase.

- id (String, cuid, PK). supabaseUserId (String, unique) — Supabase Auth user ID.
- email (String, unique). name (String, optional). phone (String, optional) — stored as '+91 <10-digit>'. avatarUrl (String, optional).
- createdAt, updatedAt.
- addresses (Address[]) — one-to-many. orders (Order[]) — one-to-many.
- Index: email. @@map('customers').

5.5 Address (addresses)

Purpose: Customer's saved delivery addresses. Each address has lat/lng + a precomputed distanceKm so the cart can show the delivery fee without re-running Haversine on every render.

- id (String, cuid, PK). customerId (String) — FK to Customer; onDelete: Cascade.
- label (String, optional) — 'Home' | 'Work' | 'Other'.
- houseFlat (String), streetArea (String), landmark (String, optional), city (String), pincode (String).
- latitude (Float), longitude (Float) — used for distance calc and Google Maps deep-links.
- distanceKm (Float, optional) — cached Haversine distance from restaurant, computed at save time.
- isDefault (Boolean, default false) — only one address can be default at a time (enforced by the API).
- createdAt, updatedAt.
- orders (Order[]) — one-to-many relation.
- Index: customerId. @@map('addresses').

5.6 Order (orders) — CENTRAL TABLE

Purpose: Customer's placed order. Holds the bill snapshot, delivery address snapshot, status timeline, payment mode + payment tracking, UPI verification state, customer rating, and the Telegram message IDs for multi-device sync.

- id (String, cuid, PK). orderNumber (String, unique) — format AB-YYYY-NNNN.
- customerId (String) — FK to Customer. addressId (String) — FK to Address.
- Snapshot fields: customerName (String), customerPhone (String, optional), addressLine (String) — composed at checkout time.
- Delivery status: status (String, default 'NEW') — values: NEW | PREPARING | OUT_FOR_DELIVERY | DELIVERED | CANCELLED.
- Checkout payment fields: paymentMode (String, default 'COD') — values: COD | UPI. paymentStatus (String, default 'PENDING') — values: PENDING | PAID.

- Admin-controlled payment tracking (separate from checkout fields): paymentReceived (Boolean, default false). paymentReceivedMethod (String, optional) — 'Cash' | 'UPI' | null. (The admin marks these when payment is actually received at the door — the method may differ from the original checkout choice.)
- UPI online payment verification flow — drives the admin 'Pending Verification' filter and the customer-facing payment screen state machine:
- upiStatus (String, default 'NONE') — values: NONE | AWAITING_SCREENSHOT | PENDING_VERIFICATION | PAID | EXPIRED.
- verifiedUtr (String, optional) — UTR/transaction reference of the SUCCESSFULLY verified payment. Used to detect duplicate/reused screenshots across orders.
- upiScreenshotUrl (String, optional) — URL of the last uploaded screenshot (for admin review when pending verification).
- upiVerificationResult (String, optional) — structured Gemini verification result JSON (for admin review).
- telegramMessageIds (String, optional) — JSON array of {chatId, messageId} pairs for the Telegram messages sent when this order entered PENDING_VERIFICATION state. Stored so that when an admin presses Verify/Reject on ONE device, the webhook can edit the message on ALL devices (multi-chat sync).
- Bill snapshot: itemTotal (Int), handlingFee (Int, default 0 — legacy), deliveryFee (Int, default 0), gstAndCharges (Int, default 0 — legacy), totalAmount (Int).
- distanceKm (Float, optional) — snapshot of the delivery distance at order time. notes (String, optional) — customer's order notes.
- rating (Int, optional) — 1-5 star rating submitted post-delivery. review (String, optional) — customer's review text.
- createdAt, updatedAt.
- items (OrderItem[]) — one-to-many. statusHistory (OrderStatusHistory[]) — one-to-many. paymentAttempts (PaymentAttempt[]) — one-to-many.
- Indexes: customerId, status, createdAt. @@map('orders').

5.7 OrderItem (order_items)

Purpose: Line items in an order. Snapshot of the menu item at order time — itemName and itemPrice are denormalized so the order history is preserved even if the menu item is later deleted or its price changes.

- id (String, cuid, PK). orderId (String) — FK to Order; onDelete: Cascade.
- itemId (String, optional) — FK to MenuItem; onDelete: SetNull (so order history is preserved if an item is deleted).
- itemName (String) — snapshot. itemPrice (Int) — snapshot. quantity (Int). imageUrl (String, optional) — snapshot.
- createdAt (DateTime, default now()).
- Index: orderId. @@map('order_items').

5.8 OrderStatusHistory (order_status_history)

Purpose: Audit log of every status change for an order. Each entry has a status string and an optional note.

- id (String, cuid, PK). orderId (String) — FK to Order; onDelete: Cascade.
- status (String) — the new status. note (String, optional) — note about the transition.
- createdAt (DateTime, default now()).
- Index: orderId. @@map('order_status_history').

5.9 PaymentAttempt (payment_attempts)

Purpose: One order can have multiple UPI payment attempts (customer retries after expiry). Each attempt tracks a single 5-minute screenshot-upload window and the Gemini Vision verification result for the screenshot uploaded in that window.

- id (String, cuid, PK). orderId (String) — FK to Order; onDelete: Cascade.
- attemptNumber (Int) — 1, 2, 3, ... — increments per order.
- status (String, default 'AWAITING_SCREENSHOT') — values: AWAITING_SCREENSHOT | PENDING_VERIFICATION | VERIFIED | REJECTED | EXPIRED.
- upiTxnRef (String, unique) — the unique transaction reference embedded in the UPI deep link (tr=...). Format: AB-YYYY-NNNN-A{attemptNumber} (e.g. AB-2026-0005-A1).
- Snapshot of the store's UPI config (so historical attempts remain valid even if env vars change later): payeeVpa (String), payeeName (String), amount (Int).
- startedAt (DateTime, default now()). expiresAt (DateTime) — startedAt + 5 min.
- Screenshot uploaded by customer: screenshotUrl (String, optional). uploadedAt (DateTime, optional).
- Gemini Vision verification result (parsed structured fields): geminiIsAuthentic (Boolean, optional), geminiExtractedTimestamp (String, optional), geminiExtractedAmount (Int, optional), geminiExtractedPayee (String, optional), geminiExtractedUtr (String, optional), geminiConfidence (Float, optional), geminiReasoning (String, optional), geminiRawResponse (String, optional).
- failedChecks (String, optional) — comma-separated list of failed checks (for admin review): e.g. 'authenticity,amount' or '' if all passed.
- verifiedAt (DateTime, optional).
- Admin manual review fields: adminReviewedBy (String, optional) — 'admin' | 'telegram'. adminReviewedAt (DateTime, optional). adminNote (String, optional).
- createdAt, updatedAt.
- Unique constraint: [orderId, attemptNumber]. Indexes: status, geminiExtractedUtr. @@map('payment_attempts').

5.10 RestaurantConfig (restaurant_config) — SINGLETON

Purpose: A single-row table (id is always 1) holding global restaurant-wide settings that the admin can toggle live from the dashboard — no redeploy required.

- id (Int, PK, default 1) — always 1 (singleton).
- isAcceptingOrders (Boolean, default true) — when false, the checkout API rejects all new orders with a 503 'orders paused' response. The customer-facing UI shows a calm 'We'll be back soon' message instead of an error.
- updatedAt (DateTime, @updatedAt).
- Comment: 'To add more settings later, add columns here. The public /api/settings route only exposes isAcceptingOrders — nothing else is publicly readable.'
- @@map('restaurant_config').

6. Third-Party Integrations

The application integrates with 8+ external services. Each is listed below with its exact role.

6.1 Turso (libSQL)

Role: Primary relational database. Accessed via `@prisma/adapters-libsql` + `PrismaClient`. Connection URL + auth token from `TURSO_DATABASE_URL` + `TURSO_AUTH_TOKEN` env vars. Holds: customers, addresses, orders, order items, menu items, categories, order status history, restaurant config, reviews, ratings, UPI payment attempts. The Prisma client is singleton-cached on `globalThis` in non-production to survive Next.js HMR.

6.2 Supabase Auth (Customer Authentication)

Role: Customer authentication only (admin uses a separate hardcoded credential check). Browser-side: cookie-persisted session via `@supabase/ssr` (storageKey: 'sb-apna-baithak-auth'). Server-side: `getSupabaseForUser(req)` reads session from cookies (set by `@supabase/ssr` browser client) and falls back to a Bearer Authorization header if present. Auth events listened to via `supabase.auth.onAuthStateChange` — invalidates auth-scoped query keys (`['addresses']`, `['orders']`, `['restaurant-rating']`) on `SIGNED_IN` and removes them on `SIGNED_OUT`. Sign-up stores `full_name` + `phone` in `user_metadata`. Sign-in uses `signInWithPassword`. Google OAuth uses `signInWithOAuth({provider:'google', redirectTo:origin/})`. Email verification flow supported via `resend({type:'signup', email})`.

6.3 Supabase Storage (Image Storage)

Role: Image storage for menu item images AND UPI payment screenshots. Two buckets used: 'menu-items' (public) for menu item images + category icons (same bucket, different path prefixes: 'menu-items/{slug-or-id}/{timestamp}-{rand6}.{ext}' and 'categories/{slug-or-id}/{timestamp}-{rand6}.{ext}'), and 'media' (public, 5MB limit) for UPI payment screenshots at 'payment-screenshots/{orderId}/{attemptId}-{timestamp}.{ext}'. Public URLs only (no signed URLs) — anyone with the URL can access the file. Image resizing: NONE — despite `sharp@0.34.3` being listed in `package.json`, `sharp` is NOT imported anywhere in the codebase.

6.4 Telegram Bot API (Admin Notifications + Inline Admin Actions)

Role: Secondary admin surface. Sends notifications to admin chat IDs (comma-separated `TELEGRAM_CHAT_IDS`, falls back to singular `TELEGRAM_CHAT_ID` for legacy compat) AND accepts `callback_query` events from inline keyboard buttons (Verify/Reject). Webhook at `/api/telegram/webhook` with `X-Telegram-Bot-API-Secret-Token` header validation against `TELEGRAM_WEBHOOK_SECRET` env var. Webhook registered for `callback_query` updates only (no message updates). Max connections: 40. Webhook registered via `scripts/register-telegram-webhook.ts`.

Three notification types:

- `notifyNewOrder(orderId)` — sent on every successful order creation. Includes: order number, customer name, phone (as text — Telegram auto-links), address, distance, lat/lng, Google Maps view link, Google Maps directions link (from restaurant to customer, driving mode), item list (bullet format with qty and line totals), total, payment mode + UPI status, customer notes.
- `notifyPaymentReceived(orderId, utr?)` — sent when Gemini auto-approves OR admin manually approves a UPI payment. Includes '₹{amount}' via UPI · UTR {utr}' line.

- `notifyPaymentPendingManual(orderId)` — sent when Gemini auto-verification FAILS or customer chose 'Continue without screenshot'. Sends the inline Verify/Reject keyboard with optional Map/Directions URL buttons. Returns the `[{chatId, messageId, ok}]` array so the caller can persist them on `order.telegramMessageIds` for later multi-device editing.

Restaurant location for directions: `process.env.RESTAURANT_LAT / RESTAURANT_LNG` with fallback `25.337698 / 82.351485`. All notifications fire non-blocking `(.catch(console.error))` — checkout never fails on Telegram outage).

6.5 Google Gemini Vision (UPI Screenshot Verification)

Role: AI-based verification of UPI payment screenshots. Model: `gemini-2.5-flash` via `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${GEMINI_API_KEY}`. Generation config: `responseMimeType='application/json', temperature=0.1, maxOutputTokens=1024`. Returns structured JSON with 7 fields: `is_authentic`, `extracted_timestamp`, `extracted_amount`, `extracted_payee`, `extracted_utr`, `confidence`, `reasoning`.

The prompt is explicitly designed to handle GPay's screenshot-blocking behavior (`FLAG_SECURE` makes GPay screenshots appear black). The prompt instructs Gemini to accept shared receipts (e.g. from GPay's Share button) as `VALID`; only flag as `inauthentic` on tampering signs. 4 verification checks then run on the response: authenticity, timestamp (± 10 min), exact amount match, payee substring match. UTR check was intentionally removed (informational only — extracted but not gated).

6.6 OpenStreetMap Nominatim (Geocoding)

Role: Free geocoding + reverse-geocoding (no API key required). Forward: `https://nominatim.openstreetmap.org/search?format=jsonv2&q=...&limit;=5&addressdetails;=1` — returns up to 5 results `{lat, lon, label}`. Reverse: `https://nominatim.openstreetmap.org/reverse?format=jsonv2&lat=...&lon;=...&zoom;=18&addressdetails;=1` — returns `display_name` string. All requests use User-Agent: 'ApnaBaithak/1.0 (apna-baithak web)' (Nominatim's usage policy requires a UA). All errors swallowed → return `"` or `[]`.

6.7 Leaflet + react-leaflet (Interactive Map)

Role: Interactive map for address selection in the customer-side Location view. Custom SVG markers (red pin for customer location, black storefront for restaurant). Draggable pin with live distance updates. Click-to-move handler. Auto-recenter on pin move (`flyTo`, duration 0.6s, max zoom 15). Auto-locate on mount via `navigator.geolocation.getCurrentPosition` (high accuracy, 8s timeout). Reverse-geocoded location auto-fills pincode (regex `\b\d{6}\b`) and city (third-from-last comma-separated part) if those fields are empty. Uses OSM tiles (cross-origin — service worker skips these).

6.8 Google Maps (Deep-Links Only — Not API)

Role: Deep-links to the public Google Maps website. NO Google Maps API key required. Two URL patterns used: `'https://www.google.com/maps/search/?api=1&query;=lat,lng'` (View on Map) and `'https://www.google.com/maps/dir/?api=1&origin;=RESTAURANT_LAT,RESTAURANT_LNG&destination;=lat,lng&travelmode;=driving'` (Get directions). These appear in Telegram messages, the admin OrderDetailModal 'Exact Location' panel, and the customer-side receipts.

6.9 Other Notable Dependencies

- Prisma ORM — schema-first ORM over Turso/libSQL. Schema at `prisma/schema.prisma`. Uses `previewFeatures: ['driverAdapters']`.

- Zustand — client-side state management with persist middleware (localStorage) for cart, favorites, app navigation, recent searches.
- qrcode package — UPI deep-link QR code generation (server-side in /api/orders/[id]/upi/initiate, NOT client-side). Options: errorCorrectionLevel 'M', margin 2, width 320px, color black-on-white.
- pdf-lib + @pdf-lib/fontkit — server-side PDF receipt generation (A4, brand letterhead PNG embedded, Poppins regular+bold fonts embedded with subsetting, ☒-glyph support, Helvetica fallback).
- Framer Motion — view-switch animations, list/card entrance animations, modal slide-up, install prompt slide-up, rating-distribution bar width animations, status-timeline pulsing ring.
- Recharts — 7-day revenue bar chart in SalesAnalytics (BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer, Cell).
- @dnd-kit/core + @dnd-kit/sortable + @dnd-kit/utilities — drag-and-drop reordering of featured items in admin FeaturedTab. PointerSensor (5px distance) + KeyboardSensor (accessibility), closestCenter collision, verticalListSortingStrategy.
- Recharts and @tanstack/react-query — server state management with configurable refetchInterval (15s for admin orders, 30s for restaurant settings, 60s for sales analytics).
- Sharp (UNUSED) — listed in package.json deps but NOT imported anywhere. Phantom dependency.
- Vercel (runtime target) — next.config.ts has output: 'standalone'; receipt-pdf.ts reads VERCEL_URL / NEXT_PUBLIC_VERCEL_PROJECT_PRODUCTION_URL env vars to HTTP-fetch its own public assets when filesystem access is unavailable (serverless).
- Caddy — reverse proxy on :81, dynamic-port routing via ?XTransformPort= query param, default upstream localhost:3000. Used by the sandbox preview system.

7. Cross-Cutting Systems

7.1 Authentication & Session Handling

The customer side and admin side use completely different auth systems:

Customer-side auth (Supabase):

- Supabase Auth via @supabase/ssr — cookie-persisted session (NOT localStorage — chosen to eliminate the 'log in again on every page reload' loop).
- Browser client: getSupabaseBrowser() — persistSession:true, autoRefreshToken:true, detectSessionInUrl:true, storageKey:'sb-apna-baithak-auth'.
- Server client: getSupabaseForUser(req) — reads session from cookies (set by browser client) and falls back to Bearer Authorization header if present.
- Service-role client: getSupabaseServer() — bypasses RLS, used for admin operations and storage uploads. SUPABASE_URL + SUPABASE_SERVICE_ROLE_KEY.
- Sign-up stores full_name + phone in user_metadata. Sign-in uses signInWithPassword. Google OAuth uses signInWithOAuth({provider:'google', redirectTo:origin/}).
- Email verification flow supported via resend({type:'signup', email}).
- AuthProvider React context: exposes {session, profile, loading, profileLoading}. Subscribes to supabase.auth.onAuthStateChange — invalidates auth-scoped query keys (['addresses'], ['orders'], ['restaurant-rating']) on SIGNED_IN and removes them on SIGNED_OUT. MountedRef, inFlightRef, lastTokenRef guards to prevent duplicate fetches.
- refreshProfile() fetches /api/auth/me with the Supabase access token. /api/auth/me auto-creates the Customer row if missing (Supabase user ↔ Customer row sync), with email-based fallback merge handling Supabase ID changes.
- updateProfile({name?, phone?}) calls PATCH /api/auth/me which uses Supabase Admin API to sync user_metadata back to Supabase.
- Soft-gating model: browsing menu, building cart, searching — all available without login. Login only required for: adding to favorites, placing order, saving address, viewing orders, viewing profile.
- authedFetch helper: transparently attaches Authorization: Bearer to every protected API call; falls back to cookie-based auth if token unavailable.

Admin-side auth (hardcoded env-var credentials + literal cookie):

- Hardcoded credentials in src/lib/constants.ts: ADMIN_CREDENTIALS = { email: process.env.ADMIN_EMAIL ?? 'admin@example.com', password: process.env.ADMIN_PASSWORD ?? 'change-me-in-env' }.
- POST /api/admin/login: receives {email, password}; lowercases+trims email; compares password as exact string. On match: returns {ok:true} and sets Set-Cookie: ab_admin=1 (httpOnly:true, sameSite:'strict', maxAge:60×60×12 = 12 hours, path:'/').
- Cookie value: literal string '1' (opaque token — NOT a JWT, NOT signed, NOT a session ID).
- Server-side guard: isAdminAuthorized(req) in src/lib/admin-guard.ts — reads ab_admin cookie and returns true iff it equals '1'. Used on EVERY admin API route.
- Client-side: AdminAuthProvider React context — pings GET /api/admin/stats on mount to verify cookie validity (any 2xx → authed). logout() clears cookie by writing max-age=0 to document.cookie, sets authed=false, calls router.refresh().
- AdminBackGuard: pushes sentinel history.pushState so Android/iOS hardware back button cannot immediately leave /admin; first back-press navigates to / (Home).
- No multi-admin support. No rate limiting. No lockout. No audit trail of which admin performed an action.

7.2 PWA Features (Customer + Admin separately)

Customer PWA (/manifest.json, scope /):

- Name: 'Apna Baithak'. Short name: 'Apna Baithak'. Description: 'your neighbourhood kitchen on Suriyawan Road. Fresh pizzas, burgers, pasta, chaat & more. Order online for delivery within 5 km.' (Note: description says 5 km, but the actual radius is 10 km.)
- Start URL: '/'. Scope: '/'. Display: 'standalone'. Orientation: 'any'. Theme color: '#FF5252' (brand red-orange). Background color: '#FF5252'. Categories: ['food', 'shopping', 'lifestyle'].
- 7 icons declared: 16x16, 32x32, 48x48, 192x192, 512x512 (purpose 'any'), 192x192, 512x512 (purpose 'maskable'). All sourced from /brand/*.
- 3 shortcuts: 'View Menu' (/view=categories), 'My Cart' (/view=cart), 'My Orders' (/view=orders) — but note: the customer app only handles ?item= deep-links; ?view= params are not actually parsed by the SPA.
- Service worker (/sw.js, scope /): precaches app shell (/ , /manifest.json, /brand/icon-192x192.png, /brand/icon-512x512.png, /brand/apple-touch-icon.png, /brand/wordmark.png) on install. Network-first for navigations (fallback to cached '/'). Stale-while-revalidate for static assets (style/script/font). Cache-first for /brand/ images. Stale-while-revalidate for other images. Skips API routes (always network). Skips cross-origin (Supabase, OpenStreetMap tiles).
- InstallPrompt component: listens for beforeinstallprompt; suppresses default mini-info bar (e.preventDefault()); shows custom branded banner 4s after event; 3-day dismiss cooldown in localStorage['apna-baithak-install-dismissed']; never shown if display-mode: standalone OR pathname starts with /admin; banner hidden in landscape; spring animation in/out.
- ServiceWorkerRegister component: production-only; skipped on /admin routes; skipped if no serviceWorker support; registers after window.load; silent failure.

Admin PWA (/admin-manifest.json, scope /admin):

- Name: 'Apna Baithak Admin'. Short name: 'AB Admin'. Description: 'manage orders, menu items, reviews and UPI verifications from your phone.'
- Start URL: '/admin'. Scope: '/admin'. Display: 'standalone'. Orientation: 'portrait'. Theme color: '#FF5252'. Background color: '#1A1A1A' (dark — blends with admin dark chrome). Categories: ['business', 'productivity', 'utilities'].
- Same 7-icon set as customer manifest but sourced from /admin-brand/*.
- 3 shortcuts: 'Pending Orders' (/admin?filter=pending), 'UPI Verifications' (/admin?filter=upi-review), 'Menu Manager' (/admin?tab=menu) — note: these query params are NOT actually parsed by the admin dashboard, which uses internal useState for tab/filter state.
- Service worker (/admin-sw.js, scope /admin): pre-caches /admin/offline.html on install. Network-first for HTML navigations with offline fallback. Cache-first for /_next/static/ and /admin-brand/ assets. Network-only for /api/* routes. Skips cross-origin. Separate cache names: 'apna-baithak-admin-v1' (navigations) + 'apna-baithak-admin-assets-v1' (static assets).
- AdminInstallPrompt component: only mounts via /admin/layout.tsx; separate localStorage dismiss key 'apna-baithak-admin-install-dismissed'; 3-day cooldown; 4s delay after beforeinstallprompt; hidden in landscape; hidden if standalone; admin-specific branding (ShieldCheck icon in red-orange gradient, 'Install Apna Baithak Admin' copy).
- AdminServiceWorkerRegister component: production-only; registers /admin-sw.js with scope '/admin/'.
- Offline fallback page (/public/admin/offline.html): self-contained HTML/CSS (no external assets). Dark background #1A1A1A, light text #FAFAFA. 96×96 red icon box with 'AB' wordmark. 'You're offline' H1. Body copy: 'The admin panel needs an active internet connection to load orders, menu items, and live data. Please reconnect and try again.' 'Retry' button → location.reload().

7.3 Notification Systems

Three distinct notification channels coexist in this app:

Telegram Bot notifications (admin):

- `notifyNewOrder(orderId)` — sent on every successful order creation. Includes: order number, customer name, phone (as text — Telegram auto-links), address, distance, lat/lng, Google Maps view link, Google Maps directions link, item list, total, payment mode + UPI status, customer notes. Header varies by payment mode (UPI → 'New Order {orderNumber}' (UPI — pending payment)).
- `notifyPaymentReceived(orderId, utr?)` — sent when Gemini auto-approves OR admin manually approves a UPI payment. Includes '{amount} via UPI · UTR {utr}' line.
- `notifyPaymentPendingManual(orderId)` — sent when Gemini auto-verification FAILS or customer chose 'Continue without screenshot'. Sends the inline Verify/Reject keyboard with optional Map/Directions URL buttons.

In-app toast notifications (both customer and admin):

- Library: 'sonner' (per package.json), used throughout. Also `shadcn/ui` toaster mounted in customer layout for backward compat.
- Customer-side toast triggers: add-to-cart, update-quantity, sign-in success, sign-out, profile update success, address save, 'Please sign in to save favorites', 'Link copied' (share), order placed, order placed + complete UPI payment, error messages.
- Admin-side toast triggers: 15+ distinct events including login success/failure, logout, status update, payment update, UPI review approve/reject, order acceptance toggle, item/category create/edit/delete, image/icon upload, export success/failure, new-order arrival (Bell icon, 6s duration), drag-reorder failure, bulk update result.
- Admin new-order toast has Bell icon and 6-second duration (longer than default) so it's noticed even when admin is glancing away.

In-app polling notifications (admin):

- Admin order polling: 15-second `refetchInterval` on `GET /api/admin/orders`. Compares fetched order IDs to a `useRef` of known IDs; toasts 'New order {orderNumber} — X' for each new one. Seeds the known-IDs set on first load so existing orders don't trigger toasts.
- Admin sales polling: 60-second `refetchInterval` on `GET /api/admin/stats/sales`. Silently refreshes the 7-day chart and top-items list.
- Customer order-acceptance polling: 30-second `refetchInterval` on `GET /api/settings`. When `isAcceptingOrders` becomes false, home view shows 'We're currently closed' banner, checkout view shows 'We'll be back soon' banner.

7.4 Real-Time / Live-Updating Features

- Live delivery-fee calculation (cart + checkout views): subtotal, delivery fee, total, and remaining-for-free-delivery all recompute reactively from cart store + selected address.
- Live distance badge (location view): updates as the user drags the pin. Within/Outside delivery-area chip flips color.
- Live reverse geocoding (location view): debounced (500ms) `reverseGeocode(pin)` after pin moves — auto-fills pincode and city if those fields are empty.
- Live countdown timer (UPI payment view): every second computes `remainingMs = attempt.expiresAt - Date.now()`; switches to expired phase when 0. Countdown banner color flips red when <60s remaining.
- Live order-acceptance status (home + checkout views): polled every 30 seconds from `/api/settings`.
- Live new-order toasts (admin dashboard): polled every 15 seconds from `/api/admin/orders`.

- Live sales analytics refresh (admin analytics tab): polled every 60 seconds from `/api/admin/stats/sales`.
- Optimistic UI updates (admin): drag-and-drop reorder, status changes, payment updates — all update the local cache instantly via `qc.setQueryData`, then sync with server.
- Auto-expire UPI attempts: `GET /api/orders/[id]/upi/status` lazily flips `AWAITING_SCREENSHOT` → `EXPIRED` on read if `now > expiresAt`.
- Order-tracking timeline animation (customer orders view): 4-step horizontal timeline with animated width 0→100% on connecting lines when step is done, animated pulse ring around the active step's circle.
- Reactive cache invalidation: `useQueryClient` used pervasively. After every mutation, the dashboard invalidates `admin-orders`, `admin-stats`, `admin-items`, `admin-categories`, `admin-featured`, `admin-menu-items`, `menu-featured` (public), `menu-items` (public), `categories` (public) — depending on what changed.

7.5 Image & Media Handling

- Menu item images: up to 5 per item (enforced in admin upload logic — `POST /api/admin/menu/items/[id]/images` checks `count < MAX_ITEM_IMAGES`). Each image has its own `sortOrder`. Stored in Supabase Storage 'menu-items' bucket at `'menu-items/{slug-or-id}/{timestamp}-{rand6}.{ext}'`. Public URL only — no signed URLs.
- Category icons: 1 per category. Stored in the SAME 'menu-items' bucket at `'categories/{slug-or-id}/{timestamp}-{rand6}.{ext}'`. Replacing an icon does NOT delete the previous one (orphaned files accumulate).
- UPI payment screenshots: stored in Supabase Storage 'media' bucket at `'payment-screenshots/{orderId}/{attemptId}-{timestamp}.{ext}'`. Ext derived from `mimeType` (png/webp/jpg). Public URL fetched back via `fetchImageAsBase64()` for Gemini Vision API call.
- Customer avatar: stored on the Supabase Auth user (`user_metadata.avatar_url`); mirrored to `Customer.avatarUrl` on `/api/auth/me`.
- Receipt PDFs: generated on-demand via `pdf-lib` + `@pdf-lib/fontkit`. A4 page (595.28 × 841.89 pt), safe zones (header 24%, footer 20%), brand letterhead PNG embedded full-page, Poppins regular+bold fonts embedded with subsetting. Multi-page support — when `cursorY` drops below `CONTENT_BOTTOM + 12`, a new page is created with the letterhead re-drawn.
- QR code: UPI deep-link QR code generated server-side via 'qrcode' package (320×320, `errorCorrectionLevel 'M'`, margin 2, black-on-white). Returned as base64 PNG data URL in the `initiate` response.
- Image transformation: NONE — no Sharp usage anywhere in the codebase (despite sharp being in `package.json`). Images stored at original uploaded size.
- DELETE on menu items: removes files from storage; DELETE/PATCH on item images does NOT remove files (only DB rows). DELETE on category does NOT remove the icon file from storage.

7.6 Admin Controls / App-Wide Toggles

- Restaurant Acceptance Toggle (`PATCH /api/admin/settings` with `{isAcceptingOrders: boolean}`) — the ONLY app-wide toggle. When false, `/api/checkout` returns 503 `ORDERS_PAUSED`; customer UI shows 'We're currently closed' banner (home) and 'We'll be back soon' banner (checkout); checkout button disabled as 'Orders paused'.
- Menu item availability toggle (`isAvailable` on `MenuItem`) — when false, hidden from public `/api/menu/items` and `/api/menu/featured` but still shown if directly linked from an order (so customers can re-view past orders).
- Category visibility toggle (`isActive` on `Category`) — when false, hidden from public `/api/menu/categories` but still visible to admins.

- Featured toggle (isFeatured on MenuItem) — controls homepage Featured rail visibility. Toggled via PATCH /api/admin/menu/featured with {itemId, isFeatured}.
- Featured order (featuredOrder on MenuItem) — controls display order within the Featured rail. Set via drag-and-drop reorder (PATCH /api/admin/menu/featured with {orderedIds: [...]}).
- Order status override (PATCH /api/admin/orders/[id] with {status}) — 5 values: NEW, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED. Each transition appends to OrderStatusHistory.
- Payment-received marking (PATCH /api/admin/orders/[id] with {paymentReceived, paymentReceivedMethod}) — admin tracks when payment was actually received at the door. Method may differ from the original checkout choice (e.g. COD order paid via UPI QR at the door).
- UPI payment approve/reject (PATCH /api/admin/orders/[id]/upi-review with {action, note?}) — admin manually resolves a PENDING_VERIFICATION UPI order. Also doable via Telegram webhook callback.
- Order delete (DELETE /api/admin/orders/[id]) — permanently deletes an order and cascades.
- Menu item delete (DELETE /api/admin/menu/items/[id]) — deletes item + its Supabase Storage images.
- Category delete (DELETE /api/admin/menu/categories/[id]) — refuses if items still assigned (409 with item count).
- Bulk order status update (admin dashboard Orders tab → Select → Mark {status}) — fires parallel PATCHes for all selected IDs.

7.7 Pricing / Business-Logic Engine

Lives in src/lib/delivery.ts — single source of truth imported by both backend and frontend. Constants: MIN_DISTANCE_KM=1, MAX_DISTANCE_KM=10, MIN_ORDER_SUBTOTAL=₹200, FAR_DISTANCE_THRESHOLD_KM=7, FAR_MIN_ORDER_SUBTOTAL=₹800, FREE_DELIVERY_OVERRIDE=₹2000, MIN_DELIVERY_CHARGE=₹20, MAX_DELIVERY_CHARGE=₹70.

- Distance serviceability: isDistanceServiceable(distKm) returns distKm <= RESTAURANT.deliveryRadiusKm (default 10 km, env-overridable).
- Order eligibility (distance-first to fix wrong-message-priority bug): if distanceKm > 7 AND subtotal < 800 → BELOW_FAR_MIN_SUBTOTAL. if subtotal < 200 → BELOW_MIN_SUBTOTAL.
- Delivery charge: if subtotal >= 2000 → finalCharge = 0 (free override). Else: raw = 20 + (clamped_distance - 1) × (70 - 20) / (10 - 1) = 20 + 5.5556 × (distance - 1). Distance clamped to [1, 10]. Rounded to nearest ₹5.
- roundToNearest5(value) = Math.round(value / 5) × 5. Sample curve: 1km → ₹20, 3km → ₹30, 5km → ₹40, 9km → ₹65, 10km → ₹70.
- remainingForFreeDelivery(distanceKm, subtotal): returns Math.max(0, 2000 - subtotal) if eligible, else null.
- Quantity clamping: each item's quantity clamped to [1, 50] via Math.max(1, Math.min(50, Number(i.quantity) || 1)).
- Order number generation: AB-YYYY-NNNN where NNNN is zero-padded to 4 digits. Sequence derived from MAX(orderNumber) + 1 for the current year (NOT COUNT(*) + 1 — survives deletions). 5-retry loop handles P2002 (unique constraint) race conditions; each retry bumps the sequence by 1.
- Handling fee + GST/charges: ₹0 each (legacy schema fields retained for historical orders; not rendered on new receipts).

7.8 Responsive / Multi-Device Handling

- Mobile (portrait): TopBar (mobile-only location chip + bell + profile), BottomNav (3-tab Home/Menu/Cart), StickyCartBar (floating above bottom nav), bottom-sheet modals on admin OrderDetailModal.

- Tablet / Desktop (landscape): TopNav (sticky top with brand mark + location chip + Home/Menu/Cart tabs + profile), no BottomNav, no StickyCartBar (TopNav cart tab is used instead), centered modals on admin OrderDetailModal.
- Responsive grids: featured items (sm:2 / md:3 / lg:4 / xl:5 / 2xl:6 cols), popular items (2/3/4/5/6 cols), category grid (sm:4 / md:6 / lg:8 cols on home; 2/3/4/6 cols on categories view), item-detail related rail (1 row scroll on mobile, grid on desktop).
- Safe area insets: pb-[env(safe-area-inset-bottom)] on BottomNav and sticky bars for iPhone notch support; pb-[calc(0.75rem+env(safe-area-inset-bottom))] on checkout Place Order button.
- useIsMobile hook: reactive boolean based on matchMedia('(max-width: 767px)') — used by shadcn/ui components internally.
- Viewport meta: themeColor #FF5252 (customer), #1A1A1A (admin); device-width, initial-scale 1, maximum-scale 1, userScalable false (prevents pinch-zoom — both customer and admin).
- No dedicated TV breakpoint — max-w-screen-2xl on the container caps content width on ultrawide displays.

7.9 Drag-and-Drop / Advanced UI Interactions

- Featured items drag-and-drop reorder (admin FeaturedTab) — @dnd-kit/core + @dnd-kit/sortable. Sensors: PointerSensor (5px activation distance to avoid accidental drag on remove button click) + KeyboardSensor (accessibility, with sortableKeyboardCoordinates). Collision: closestCenter. Strategy: verticalListSortingStrategy. Visual feedback while dragging: border-brand, elevated shadow, opacity 0.8, z-index 10. Optimistic update via qc.setQueryData; PATCH full orderedIds array; failure reverts via invalidate.
- Leaflet draggable map pin (customer Location view) — draggable:true on Leaflet Marker; updates on drag (live) and dragend (final); live distance label updates as the user drags.
- Map click-to-move pin — map-click handler moves the pin to the clicked latlng.
- Auto-recenter on pin move — map.flyTo(center, max(currentZoom, 15), {duration:0.6}) whenever pin position changes.
- Carousel (ImageGallery) — AnimatePresence slide animation (x: 30 → 0 → -30), Prev/Next chevron buttons, page indicator pill, dot indicators, fullscreen Lightbox with scale-in animation.
- Bottom-sheet modals (admin OrderDetailModal, customer EditProfileModal) — bottom-sheet on mobile (place-items-end, rounded top corners), centered modal on desktop (place-items-center, fully rounded).
- Hardware back-button integration (customer + admin) — pushes history state on view changes; popstate listener calls back() or routes to /.
- Swipe-back gesture: NOT implemented — relies on native browser back gesture / hardware back button.
- Category sort reorder does NOT use dnd-kit — uses simple up/down ChevronUp/ChevronDown arrow buttons that swap sortOrder between two categories via parallel PATCH calls.

7.10 AI / Automation Features

- UPI screenshot verification via Google Gemini Vision (gemini-2.5-flash) — the only AI feature in the codebase. Customer uploads a screenshot of their UPI payment; the screenshot is sent to Gemini with a structured prompt; Gemini returns JSON with 7 fields (is_authentic, extracted_timestamp, extracted_amount, extracted_payee, extracted_utr, confidence, reasoning). 4 verification checks then run: authenticity, timestamp (± 10 min), exact amount match, payee substring match. If all 4 pass → auto-verify order as PAID. If any check fails → mark as PENDING_VERIFICATION and notify admin via Telegram with inline Verify/Reject buttons.

- Prompt explicitly handles GPay's screenshot-blocking behavior (FLAG_SECURE) — GPay renders screenshots as black images. The prompt instructs Gemini to accept shared receipts (e.g. from GPay's Share button → Save/Download) as VALID; only flag as inauthentic on tampering signs. This is documented in code comments.
- UTR (Unique Transaction Reference) extraction — Gemini extracts the UTR from the screenshot. The UTR is stored on the PaymentAttempt for admin review. However, UTR is NOT used as a pass/fail gate (the check was removed because GPay blocks screenshots — extracting UTR from a shared receipt isn't reliable). UTR is informational only.
- Duplicate UTR detection — schema has a verifiedUtr field on Order (designed for cross-order UTR reuse detection) and an index on PaymentAttempt.geminiExtractedUtr, but the actual cross-order UTR reuse check is NOT implemented in current code. This is a known regression — comments document the removal.
- No other AI features — no AI-powered recommendations, no AI search, no AI chatbot, no AI-generated content. The Gemini integration is strictly scoped to UPI screenshot verification.

7.11 Zustand Stores (Client State)

Four Zustand stores with persist middleware (localStorage):

Store	localStorage key	State + Actions
useCart	apna-baithak-cart	lines: CartLine[] = {itemId, name, slug, price, imageUrl, quantity}. Actions: addItem(item, qty=1), removeItem(itemId), setQty(itemId, qty), increment, decrement, clear. Selectors: count() (sum of quantities), subtotal() (sum of price × qty).
useFavorites	apna-baithak-favorites	favorites: FavoriteItem[] = {id, name, slug, price, imageUrl}. Actions: toggle(item), isFavorite(id), count(), clear().
useApp	apna-baithak-app	view: View (12 views); activeCategorySlug, activeItemSlug, selectedAddressId, search; _stack: View[] (capped 30); loginReturnTo: View null; activeOrder: Order null; upiConfirmationStatus: 'PAID'/'PENDING_VERIFICATION' null. Actions: setView, openCategory, openItem, setSelectedAddressId, setSearch, goToLogin(returnTo?), returnFromLogin, goToUpiPayment(order), goToOrderConfirmation(order, upiStatus) — TRIMS checkout+upi-payment from stack, back(). Persistence: partialize — only selectedAddressId, activeOrder, upiConfirmationStatus are persisted.
useRecentSearch	apna-baithak-recent-search	recent: string[]. Actions: add(query) — trims, ignores <2 chars, dedupes case-insensitive, prepends, caps at 8 entries; clear(); remove(query).

7.12 Scripts Directory (Maintenance Utilities)

11 standalone TypeScript scripts under /scripts — diagnostic, migration, and verification utilities:

- diagnose-checkout.ts — Turso DB diagnostic: counts orders this year, computes next order number, checks for unique-constraint collisions, lists all orders, runs a dry-run order.create() in a rolled-back transaction to reproduce the exact error.
- create-restaurant-config.ts — Creates the restaurant_config table on Turso (singleton row id=1, isAcceptingOrders=true) via direct @libsql/client (NOT Prisma).
- add-featured-columns.ts — Adds isFeatured + featuredOrder columns to menu_items table via direct ALTER TABLE.
- check-turso-schema.ts — Checks if the orders table has the telegramMessageIds column; adds it if missing (direct ALTER TABLE). Required for the multi-chat Telegram message-sync feature.

- `verify-delivery-charge.ts` — Test suite for the delivery pricing system. Validates reference values (1km/₹300→₹20, 3km/₹250→₹30, 5km/₹300→₹40, 9km/₹850→₹65, 10km/₹1000→₹70, 3km/₹150→BLOCKED, 9km/₹500→BLOCKED, 9km/₹2100→₹0).
- `test-gemini-api.ts` — Smoke-test the Gemini 2.5 Flash API with a 1×1 red-pixel PNG to verify the model name, API key, and structured-JSON output mode all work.
- `register-telegram-webhook.ts` — One-shot script to register the Telegram bot webhook at `{WEBHOOK_URL}/api/telegram/webhook` with `secret_token`, `allowed_updates: ['callback_query']`, `max_connections: 40`. Run after each deployment.
- `verify-bug-fix-messaging.ts` — Test suite for two cart messaging fixes: (1) distance-first eligibility (9.71 km / ₹150 cart shows the ₹800 message, not ₹200); (2) delivery fee always shown independently of eligibility.
- `verify-location-radius.ts` — Test suite for the location picker: generates lat/lng points at specific distances (1, 3, 5, 6, 7, 8, 9.5, 10, 10.01, 11, 15 km) directly north of the restaurant, verifies `distanceFromRestaurant`, `isWithinDeliveryRadius`, and the badge text.
- `verify-bug-fix.ts` — Test suite for the delivery-fee inversion bug: confirms orders under ₹200 are BLOCKED (not given free delivery), and that the ₹2000+ override still works correctly.
- `verify-order-number-fix.ts` — Read-only DB check that compares the OLD order-number logic (COUNT + 1, buggy due to deleted-order gaps) vs the NEW logic (MAX + 1 via `findFirst` with `orderBy: orderNumber desc`).

8. Full Flat Feature List

Below is the deduplicated, flat, numbered list of every distinct feature identified anywhere above. Each item is a single, countable capability — a small toggle switch counts as much as the entire checkout flow. Use this list directly for development-cost estimation: count the items, multiply by your per-feature rate, and you have the rebuild estimate.

- 1. Customer PWA installable from / (own manifest, /sw.js service worker, /brand/* icon set).
- 2. Customer PWA theme color #FF5252, standalone display, any orientation.
- 3. Customer PWA shortcuts (3): View Menu, My Cart, My Orders (manifest entries; not actually parsed by SPA).
- 4. Customer PWA install prompt with 4-second delay, 3-day dismiss cooldown (localStorage), branded banner, never shown in landscape or if already standalone or on /admin routes.
- 5. Customer service worker precaches app shell (/ , manifest, brand icons), network-first for navigations, stale-while-revalidate for static assets, cache-first for /brand/ images, stale-while-revalidate for other images, skips API routes, skips cross-origin.
- 6. Single-page-app view-routing state machine (12 views: home, categories, category-listing, item-detail, cart, location, profile, orders, checkout, login, upi-payment, order-confirmation).
- 7. Browser history integration: history.pushState on every view change; popstate listener calls back(); Android/iOS hardware back button walks in-app stack before leaving website.
- 8. Scroll-to-top on view change via requestAnimationFrame (window, documentElement, body,).
- 9. Deep-link via ?item= URL param (parses, calls openItem(slug), strips param via history.replaceState).
- 10. useMounted hook (useSyncExternalStore) to suppress cart-dependent UI during SSR hydration.
- 11. Framer Motion AnimatePresence view-switch animation (opacity + y-translate, 180ms easeOut).
- 12. TopBar (mobile portrait only) — location chip, notifications bell, profile button; hidden in landscape.
- 13. TopNav (landscape only) — brand mark, location chip, Home/Menu/Cart tabs, profile button; hidden in portrait.
- 14. BottomNav (portrait only) — 3-tab Home/Menu/Cart with active-tab highlight and cart badge.
- 15. StickyCartBar (portrait only) — floating brand-bg pill above bottom nav when cart has items; spring animation in/out; click navigates to cart.
- 16. 4px bottom spacer when cart has items (portrait only) — reserves space below floating cart bar.
- 17. suppressHydrationWarning on root + main + each BottomNav tab button (tolerates Zustand persist hydration).
- 18. Home view: search entry button (navigates to categories with autofocus).
- 19. Home view: live order-acceptance polling every 30s from /api/settings; calm blue 'We're currently closed' banner when paused.
- 20. Home view: categories rail (horizontal scroll on mobile, responsive grid 4/6/8 cols on larger).
- 21. Home view: category loading skeletons (8 skeletons, different layout mobile vs desktop).
- 22. Home view: hero promo card with gradient, restaurant name, headline, delivery ETA chip, 10-km radius chip.
- 23. Home view: average rating chip (fetched from /api/ratings) shown only if avgRating > 0.
- 24. Home view: featured items rail (horizontal scroll on mobile, responsive grid 2/3/4/5/6 cols) — entirely hidden if 0 featured items.
- 25. Home view: 'See all →' button on featured section (navigates to categories).
- 26. Home view: featured items loading skeleton (×4 mobile, ×8 desktop).
- 27. Home view: popular items section (items.slice(6, 20), responsive grid 2/3/4/5/6 cols).
- 28. Home view: popular items loading skeleton ×6.
- 29. Home view: restaurant info footer card with MapPin avatar, BrandWordmark image, full address, phone number.
- 30. Home view: CategoryButton sub-component with motion.button scale-in animation (delay capped at 0.4s).
- 31. Categories view: sticky header with back button + 'Menu' title + autofocus search input (pill-shaped).
- 32. Categories view: live search (calls /api/menu/items?search= via authedFetch, enabled only when query non-empty).
- 33. Categories view: clear (X) button on search input when query non-empty.
- 34. Categories view: recent searches section (pills from useRecentSearch Zustand store, AnimatePresence enter/exit).
- 35. Categories view: 'Clear' button on recent searches (trash icon, wipes list).

- 36. Categories view: popular searches section (static pill list: Pizza, Burger, Pasta, Maggie, Chaat, Fries).
- 37. Categories view: search-results mode (hides categories grid, shows 'N result(s) for ' header, renders ItemRows in grid 1/2/3/4 cols).
- 38. Categories view: empty-search-results state (dashed bordered card with hint to try pizza/burger/chaat).
- 39. Categories view: categories grid (default mode, 2/3/4/6 cols, image-or-fallback, name, itemCount).
- 40. Categories view: auto-record recent searches (records query if length >=2 AND items found AND not already recorded).
- 41. Category listing view: sticky header with back button, category name, item-count badge.
- 42. Category listing view: category description paragraph (only if description exists).
- 43. Category listing view: items grid (1/2/3/4 cols) fetched from /api/menu/items?category=.
- 44. Category listing view: loading skeleton (4 × ItemRowSkeleton).
- 45. Category listing view: empty state with Package icon ('No items in this category yet / Please check back soon.').
- 46. Item detail view: ImageGallery (single image OR multi-image carousel) with UtensilsCrossed fallback.
- 47. Item detail view: top gradient overlay for button legibility.
- 48. Item detail view: back button (top-left, white circle, blurred bg).
- 49. Item detail view: share button (top-right) — navigator.share first, clipboard fallback + toast.
- 50. Item detail view: content card slides up over the hero (-mt-6 rounded-t-3xl).
- 51. Item detail view: veg indicator (square border + dot, emerald-600).
- 52. Item detail view: category label (uppercase, tracked, muted).
- 53. Item detail view: inline qty selector (pill with Minus/Plus, local qty state, min 1).
- 54. Item detail view: price (2xl extrabold brand color, 'per piece' suffix).
- 55. Item detail view: star rating badge (amber, if rating > 0).
- 56. Item detail view: prep-time badge (blue, 'X-(X+10) mins').
- 57. Item detail view: calories badge (orange, 'N kcal' with Flame icon).
- 58. Item detail view: description section ('About this dish').
- 59. Item detail view: restaurant note card (brand-softer bg, Store icon, 'Orders cannot be cancelled once preparation begins.').
- 60. Item detail view: related items rail ('More from ') — horizontal scroll of up to 6 items from same category.
- 61. Item detail view: sticky bottom bar with total preview + CTA.
- 62. Item detail view: CTA label logic ('Add to cart' vs 'Add N more · ' based on inCart presence).
- 63. Item detail view: toast on add/update.
- 64. Cart view: sticky header with back button, 'Your Cart' title, item-count badge.
- 65. Cart view: empty-cart state (dashed card, ShoppingBag icon, 'Browse menu →' CTA + 'search for a specific dish' secondary link).
- 66. Cart view: address confirmation card (Instamart-style, brand-softer bg, MapPin avatar, 'Delivering to' / 'Add address to proceed' label, full address, distance, 'Change' / arrow button).
- 67. Cart view: cart line items list (image, name, '₹X each', 'Remove' button, QtyStepper sm, line total).
- 68. Cart view: '+ Add more items' link (navigates home).
- 69. Cart view: live delivery-pricing message card (emerald when free delivery unlocked, amber when blocked, brand-softer when eligible with progress).
- 70. Cart view: no-address-selected state (muted card 'Select a delivery address to see the delivery charge.').
- 71. Cart view: Bill Details card (Item Total / Delivery Fee or 'Select address' or 'FREE' / To Pay).
- 72. Cart view: non-cancellable note (amber card, ShieldCheck icon, cancellation & refund policy).
- 73. Cart view: sticky checkout bar with delivery-to + total + Place Order/Sign in to order button.
- 74. Cart view: checkout() function logic (auth check, address check, navigate to checkout).
- 75. Cart view: client-side delivery fee calculation mirrors the server (deliveryChargeForDistance, checkOrderEligibility, remainingForFreeDelivery).
- 76. Checkout view: sticky header with back button + 'Checkout' title.
- 77. Checkout view: phone-required guard (localStorage flag, error toast, navigate to profile which auto-opens EditProfileModal).
- 78. Checkout view: delivery address section (chosen address card with 'Change' button, or dashed 'Add a delivery address' prompt).
- 79. Checkout view: payment mode section (COD + UPI selectable cards with radio-style indicator).
- 80. Checkout view: order notes textarea (optional, 'Less spicy, call on arrival...').

- 81. Checkout view: Bill Details card (Item Total, Delivery Fee, To Pay).
- 82. Checkout view: calm 'We'll be back soon' banner when orders paused (blue card with Clock icon).
- 83. Checkout view: hard-block banner when ineligible AND accepting orders (amber 'Checkout blocked' card).
- 84. Checkout view: policy note ('orders cannot be cancelled or refunded once preparation begins').
- 85. Checkout view: sticky Place Order button (disabled when placing/auth-loading/no-phone/no-address/empty-cart/blocked/paused).
- 86. Checkout view: button label by state (Placing order..., Orders paused, Checkout blocked, Pay ☒ for UPI, Place Order · ☒ otherwise).
- 87. Checkout view: placeOrder() POST /api/checkout; on 503 silently invalidates settings query; on success routes to UPI payment or order confirmation.
- 88. Checkout view: order-acceptance polling every 30s.
- 89. Orders view: sticky header with back button + 'My Orders' title.
- 90. Orders view: loading state (2 × OrderCardSkeleton).
- 91. Orders view: empty state (dashed card, Package icon, 'No orders yet', 'Start ordering' CTA).
- 92. Orders view: OrderTrackingCard with expandable header (icon circle, order number, StatusBadge, item-count, datetime, total, payment mode).
- 93. Orders view: 5 status colors (NEW=blue, PREPARING=amber, OUT_FOR_DELIVERY=purple, DELIVERED=emerald, CANCELLED=red).
- 94. Orders view: live tracking timeline (4-step horizontal timeline for active orders only; animated connecting lines; pulsing ring on active step; step labels + descriptions).
- 95. Orders view: ETA estimate (~max(0, 35 - elapsedMin) min badge).
- 96. Orders view: delivered state (emerald bar 'Delivered on ').
- 97. Orders view: cancelled state (red bar 'This order was cancelled.').
- 98. Orders view: expanded details (items list, delivery address block, bill summary, action buttons row).
- 99. Orders view: reorder action (iterates items, calls addItem, navigates to cart, toast; skips null itemId).
- 100. Orders view: receipt download (fetches /api/orders/<id>/receipt as blob, creates object URL, programmatically clicks hidden <a download>, revokes URL after 60s; uses fetch(credentials:include) to avoid mobile popup-blocking).
- 101. Orders view: receipt button disabled until delivered, shows 'Receipt available after delivery' tooltip.
- 102. Orders view: rating section for delivered orders (display mode if rated, interactive OrderRating if not).
- 103. Order confirmation view: clears cart on mount.
- 104. Order confirmation view: sticky header with back button that goes directly to home.
- 105. Order confirmation view: status hero (PAID — emerald CheckCircle2 'Order Confirmed!'; PENDING_VERIFICATION — amber AlertTriangle 'Payment Under Review').
- 106. Order confirmation view: order summary card (items count, amount, payment badge).
- 107. Order confirmation view: delivery address card (full address, distance + ETA).
- 108. Order confirmation view: pending verification info box (4 bullet points about manual review).
- 109. Order confirmation view: action buttons ('Track my order' → orders; 'Back to home' → home).
- 110. Order confirmation view: help footer (phone + email).
- 111. UPI payment view: 5-phase state machine (checking, initiating, awaiting_payment, verifying, expired).
- 112. UPI payment view: order summary card (order number + amount to pay).
- 113. UPI payment view: mount guard (status check → auto-redirect to confirmation for PAID/PENDING_VERIFICATION).
- 114. UPI payment view: initiateAttempt() (POST /api/orders//upi/initiate; auto-opens UPI deep link).
- 115. UPI payment view: countdown timer (every second, MM:SS format, red banner when <60s).
- 116. UPI payment view: QR code card (256×256 image, payee name + VPA, UPI transaction ref).
- 117. UPI payment view: 'Pay ☒ via UPI app' button (re-opens deep link).
- 118. UPI payment view: GPay-specific guidance (blue card — screenshots are blocked in GPay).
- 119. UPI payment view: PhonePe/Paytm/BHIM guidance (emerald card).
- 120. UPI payment view: upload section (hidden file input, image type + ≤5MB validation, POST FormData).
- 121. UPI payment view: on VERIFIED outcome → toast + goToOrderConfirmation(PAID).
- 122. UPI payment view: on PENDING_VERIFICATION outcome → toast + goToOrderConfirmation(PENDING_VERIFICATION).

- 123. UPI payment view: upload button label switches to 'Verifying with AI...' during verify.
- 124. UPI payment view: 'I have paid — Continue without screenshot' button (confirm() dialog, POST /api/orders/upi/manual).
- 125. UPI payment view: helper note about verification flow.
- 126. UPI payment view: expired state (red X icon, 'Payment Window Expired', retry button).
- 127. UPI payment view: policy footer.
- 128. UPI payment view: header back button goes to orders view (not back to checkout).
- 129. Login view: back button + brand icons (BrandIcon 72px + BrandWordmark 32px).
- 130. Login view: signin/signup heading + subtitle ('Browse the menu freely without logging in.').
- 131. Login view: 'Continue with Google' button (white card, Chrome icon, OAuth via Supabase).
- 132. Login view: OR divider.
- 133. Login view: email/password form (Full Name + Contact Number fields in signup mode).
- 134. Login view: Indian mobile normalization (strips 91/0 prefixes, validates ^[6-9]\d{9}\$).
- 135. Login view: signup stores phone as +91 <10-digit> in Supabase user_metadata.
- 136. Login view: signin flow (signInWithPassword, toast 'Signed in', returnFromLogin).
- 137. Login view: signup flow (signUp with metadata, if no session returned → verification sub-screen).
- 138. Login view: verification-sent sub-screen (CheckCircle2, 'Verify your email', info box, 'Resend verification email' button).
- 139. Login view: mode toggle ('Don't have an account? Sign up' / 'Already have an account? Sign in').
- 140. Login view: error handling (toasts error messages on any failure).
- 141. Profile view: auto-open EditProfileModal if localStorage.apna-baithak-require-phone === '1'.
- 142. Profile view: loading state (centered spinner + 'Loading your profile...').
- 143. Profile view: not-signed-in state (UserIcon avatar, 'Sign in' CTA).
- 144. Profile view: profile section (avatar, name/email, phone).
- 145. Profile view: Edit Profile button (brand bg, Pencil icon).
- 146. Profile view: Logout button (outlined, LogOut icon; clears cart, toasts 'Signed out', navigates home).
- 147. Profile view: Account section (Saved Address row, Payment Modes row).
- 148. Profile view: Favorites section (image, name, price, ADD button, Remove-from-favorites button).
- 149. Profile view: My Orders section ('View all →' link + first 3 orders as compact cards, or empty state).
- 150. Profile view: Restaurant section (card with name, address, phone, email).
- 151. Profile view: EditProfileModal (bottom-sheet mobile, centered dialog desktop; name + phone + email inputs; Indian mobile normalization; Save/Cancel buttons).
- 152. Location view: lazy dynamic import with ssr:false + loading placeholder.
- 153. Location view inner: custom Leaflet markers (red SVG pin for customer, black SVG storefront for restaurant).
- 154. Location view inner: DraggablePin sub-component (live updates on drag, final on dragend).
- 155. Location view inner: ClickToMove sub-component (map-click moves pin).
- 156. Location view inner: Recenter sub-component (map.flyTo on pin move).
- 157. Location view inner: sticky header with back button + 'Select Your Location' title + search input.
- 158. Location view inner: search-as-you-type (geocode(q) + scrollable results dropdown).
- 159. Location view inner: live distance badge + Within/Outside delivery-area chip.
- 160. Location view inner: re-center floating button (top-right of map).
- 161. Location view inner: auto-locate on mount (navigator.geolocation.getCurrentPosition, high accuracy, 8s timeout).
- 162. Location view inner: reverse geocoding (debounced 500ms, auto-fills pincode + city if empty).
- 163. Location view inner: action buttons (2-col grid: Use Current Location + Add New Address).
- 164. Location view inner: Saved Addresses section (loading skeletons, list with label/address/distance/actions, selected highlight).
- 165. Location view inner: EditAddressForm inline (label selector, fields, Save/Cancel).
- 166. Location view inner: setDefault action (PATCH /api/addresses/).
- 167. Location view inner: deleteAddress action (confirm() + DELETE).
- 168. Location view inner: address form (label selector pills, required fields, 'Outside delivery area' warning, save disabled when outside radius or saving).
- 169. Location view inner: saveAddress() validates auth/fields/withinRadius; POST /api/addresses; invalidates ['addresses'] cache; toast 'Address saved'; navigate to cart.
- 170. ItemCard: motion.div with opacity+y enter animation.
- 171. ItemCard: image area (aspect-square, ShimmerImage placeholder, group-hover scale-110 zoom, UtensilsCrossed fallback).

- 172. ItemCard: veg indicator (top-left).
- 173. ItemCard: HeartButton (top-right, white/90 backdrop) — toggles favorite, auth-gated.
- 174. ItemCard: rating badge (bottom-left, black/75 backdrop, amber star) if rating > 0.
- 175. ItemCard: title (line-clamp-1), description (line-clamp-2), price + ADD/QtyStepper switch.
- 176. ItemRow: motion.div with opacity+x enter animation, 80×80 image, click opens item detail.
- 177. QtyStepper: 3 sizes (sm/md/lg), pill container, Minus/Plus buttons, e.stopPropagation, aria-labels.
- 178. ImageGallery: empty → UtensilsCrossed; single → tap to open Lightbox; multi → AnimatePresence carousel with Prev/Next chevrons, page indicator, dot indicators, expand button; Lightbox full-screen with scale-in animation.
- 179. OrderRating: pre-submit state (5 interactive stars + review textarea + Submit button); post-submit state (emerald card with display stars).
- 180. Skeletons: ItemCardSkeleton, ItemRowSkeleton, CategorySkeleton, CategoryGridSkeleton (8-cell), OrderCardSkeleton, CartLineSkeleton, BillSummarySkeleton (5-row), AddressRowSkeleton, AdminOrderRowSkeleton, StatCardSkeleton, AdminItemRowSkeleton, LoadingSpinner.
- 181. BrandWordmark + BrandIcon (unoptimized Next Image for instant load).
- 182. Providers: QueryClient (staleTime 60s, refetchOnWindowFocus false, retry 1) + AuthProvider + authedFetch helper.
- 183. AuthProvider: Supabase session + customer profile context; onAuthStateChange listener; refreshProfile; signInWithEmail; signUpWithEmail; signInWithGoogle; signOut; updateProfile.
- 184. Admin PWA installable from /admin (own /admin-manifest.json, /admin-sw.js service worker scope /admin/, /admin-brand/* icon set).
- 185. Admin PWA theme color #FF5252, background #1A1A1A (dark), portrait orientation.
- 186. Admin PWA shortcuts (3): Pending Orders, UPI Verifications, Menu Manager (manifest entries; not actually parsed by admin dashboard).
- 187. Admin PWA install prompt with 4-second delay, 3-day dismiss cooldown (separate localStorage key), branded banner with ShieldCheck icon, hidden in landscape or if standalone.
- 188. Admin service worker: pre-caches /admin/offline.html; network-first for navigations with offline fallback; cache-first for /_next/static/ and /admin-brand/ assets; network-only for /api/* routes; skips cross-origin; separate cache names from customer SW.
- 189. Admin offline fallback page (/admin/offline.html): self-contained HTML/CSS, dark #1A1A1A bg, 96×96 red 'AB' icon, 'You're offline' H1, body copy, 'Retry' button.
- 190. Admin route /admin: server-rendered entry; wraps in AdminAuthProvider; AdminBackGuard prevents hardware back-button from immediately leaving /admin; AdminGate conditionally shows Login/Dashboard/spinner.
- 191. Admin layout: separate metadata (title template '%s · Apna Baithak Admin'), admin manifest, 5 admin icons, admin theme color #1A1A1A, viewport locked (no pinch-zoom).
- 192. AdminAuthProvider: Context with {authed, login, logout, loading}; pings GET /api/admin/stats on mount to verify cookie; logout clears cookie + router.refresh().
- 193. AdminLogin: brand icons + email/password inputs + Sign in button; HTML5 validation; busy spinner; toast on success/failure.
- 194. AdminDashboard: sticky header with brand icon, wordmark, 'Restaurant owner dashboard' subtitle.
- 195. AdminDashboard: Order Acceptance Toggle button (green 'Accepting Orders' / red 'Orders Paused'; mobile 'Open'/'Paused'; pulses while toggling; PATCHes /api/admin/settings; invalidates cache; toasts success).
- 196. AdminDashboard: Logout button.
- 197. AdminDashboard: 4-card stats grid (Today's Orders, Today's Revenue, Active Orders, Total Revenue; responsive 2-col mobile / 4-col desktop).
- 198. AdminDashboard: StatCardSkeleton while stats loading.
- 199. AdminDashboard: horizontally scrollable tab bar (5 tabs: Orders, Menu, Featured, Analytics, Reviews).
- 200. AdminDashboard: live new-order polling every 15s (compares IDs, toasts 'New order {orderNumber}' — 🔔 with Bell icon 6s).
- 201. AdminDashboard: seeds known-IDs set on first load (existing orders don't trigger toasts).
- 202. AdminDashboard: updateStatus(id, status) function (PATCH /api/admin/orders/:id).
- 203. AdminDashboard: updatePayment(id, paymentReceived, method?) function.
- 204. AdminDashboard: upiReview(id, action, note?) function.
- 205. AdminDashboard Orders Tab: status filter pills (All, New, Preparing, Out for Delivery, Delivered, Cancelled).

- 206. AdminDashboard Orders Tab: '△ Pending UPI Verification' toggle pill.
- 207. AdminDashboard Orders Tab: 'Select' button (toggles bulk mode).
- 208. AdminDashboard Orders Tab: bulk action bar (n selected + per-status mark buttons + Cancel Orders + Clear).
- 209. AdminDashboard Orders Tab: Select all / Deselect all bar.
- 210. AdminDashboard Orders Tab: per-order checkbox selection toggle.
- 211. AdminDashboard Orders Tab: bulkUpdateStatus(status) fires parallel PATCHes.
- 212. AdminDashboard Orders Tab: Order cards list (animated with framer-motion stagger).
- 213. AdminDashboard Orders Tab: OrderDetailModal rendered conditionally.
- 214. AdminDashboard Orders Tab: local mutation of selected order after actions (no refetch needed).
- 215. AdminDashboard Orders Tab: OrderCard sub-component (animated entrance, status-colored icon, order number, badges, customer info, total + payment mode, ChevronRight/selection circle, quick Cash/UPI/Mark Pending payment footer).
- 216. AdminDashboard Menu Tab: sub-tab toggle (Items | Categories).
- 217. AdminDashboard Menu Tab Items: header with count + 'Add item' button; list with thumbnail, name, veg indicator, HIDDEN badge, category/price/images info, edit/delete buttons; empty state; loading skeleton.
- 218. AdminDashboard Menu Tab Items: ItemEditor modal (name, price, category, description, prep time, calories, veg, available, up to 5 images with upload/remove).
- 219. AdminDashboard Menu Tab Items: ItemEditor fetches existing item data via useQuery then populates form via useEffect.
- 220. AdminDashboard Menu Tab Items: ItemEditor after creating calls window.location.reload() to switch into edit mode.
- 221. AdminDashboard Menu Tab Categories: CategoriesManager (header with count + Add button; sortable list with up/down arrow buttons; icon, name, HIDDEN badge, 'Order: N · M items'; edit/delete; loading skeletons; empty state; delete blocked if items assigned).
- 222. AdminDashboard Menu Tab Categories: CategoryEditor modal (1:1 icon preview + Upload icon button, name input, description textarea, display order numeric, active checkbox, Save/Cancel).
- 223. AdminDashboard Menu Tab Categories: CategoryEditor disabled icon upload until category saved.
- 224. AdminDashboard Featured Tab: drag-to-reorder featured items list (DndContext + SortableContext, vertical strategy, PointerSensor 5px distance, KeyboardSensor).
- 225. AdminDashboard Featured Tab: drag handle (GripVertical, touch-none, cursor-grab), position number badge, thumbnail, name + price, Remove (X) button.
- 226. AdminDashboard Featured Tab: visual feedback while dragging (border-brand, elevated shadow, opacity 0.8, z-index 10).
- 227. AdminDashboard Featured Tab: 'Saving...' indicator while reorder in flight.
- 228. AdminDashboard Featured Tab: optimistic cache update on drag end (qc.setQueryData); PATCH full orderedIds array; failure reverts via invalidate.
- 229. AdminDashboard Featured Tab: All Menu Items list with star toggle (amber when featured, muted when not).
- 230. AdminDashboard Featured Tab: toggleFeatured(itemId, currentlyFeatured) — PATCH /api/admin/menu/featured; invalidates admin-featured + admin-menu-items + menu-featured (public homepage cache).
- 231. AdminDashboard Featured Tab: empty state when 0 featured.
- 232. AdminDashboard Analytics Tab: 7-day revenue chart card (Recharts BarChart, today highlighted in brand red, custom tooltip, ☒ formatter, rounded top corners).
- 233. AdminDashboard Analytics Tab: 7-day total revenue + total orders summary.
- 234. AdminDashboard Analytics Tab: today revenue callout.
- 235. AdminDashboard Analytics Tab: Top Selling Items card (last 30 days; rank, thumbnail, name, ☒ revenue, qty-progress-bar, qty sold).
- 236. AdminDashboard Analytics Tab: auto-refresh every 60s (refetchInterval).
- 237. AdminDashboard Analytics Tab: Export Orders (CSV) button (fetches /api/admin/export?type=orders as blob, downloads with Content-Disposition filename or fallback).
- 238. AdminDashboard Analytics Tab: Export Revenue (30-day CSV) button.
- 239. AdminDashboard Reviews Tab: 4-card summary grid (Avg Rating, Total Reviews, Rating Distribution with 5 animated bars).
- 240. AdminDashboard Reviews Tab: customer review list (animated entrance, name + order number, subtitle, 5-star display, optional review text).

- 241. AdminDashboard Reviews Tab: empty state with explanation ('Customer ratings will appear here once orders are delivered and rated.').
- 242. AdminDashboard Reviews Tab: read-only — no moderation features.
- 243. AdminDashboard OrderDetailModal: animated entrance (slide up mobile, fade desktop); backdrop click closes.
- 244. AdminDashboard OrderDetailModal: header (order number, status badge, formatted date-time, close button).
- 245. AdminDashboard OrderDetailModal: customer card (avatar, name, phone, Call/SMS action buttons if phone exists).
- 246. AdminDashboard OrderDetailModal: delivery address block (address line, distance).
- 247. AdminDashboard OrderDetailModal: Exact Location panel (when lat/lng present; lat/lng readout, View on map link, Get directions link).
- 248. AdminDashboard OrderDetailModal: Status Timeline (4-step horizontal, NOT shown for CANCELLED; animated pulsing ring on active step).
- 249. AdminDashboard OrderDetailModal: Cancelled banner.
- 250. AdminDashboard OrderDetailModal: Items list (image, name, qty × price, line total).
- 251. AdminDashboard OrderDetailModal: Bill Details (Item Total, Delivery Fee or FREE, Distance, Total, payment mode + status).
- 252. AdminDashboard OrderDetailModal: Payment Received panel (state badge, Received as Cash/UPI buttons, Reset to Pending button).
- 253. AdminDashboard OrderDetailModal: Customer Note card (amber, only if order.notes present).
- 254. AdminDashboard OrderDetailModal: UPI Payment Verification section (status pill, verified UTR, screenshot thumbnail, AI Verification Result panel with extracted fields + reasoning + failed checks list).
- 255. AdminDashboard OrderDetailModal: admin review UI (admin note text input, Approve/Reject buttons).
- 256. AdminDashboard OrderDetailModal: Print Receipt button (fetches PDF as blob, triggers <a download>.click()).
- 257. AdminDashboard OrderDetailModal: Delete Order button (confirm() + DELETE + reload).
- 258. AdminDashboard OrderDetailModal: 'Advance to next status →' button (hidden for DELIVERED/CANCELLED).
- 259. AdminDashboard OrderDetailModal: status override buttons (one per non-current status; CANCELLED styled red).
- 260. AdminDashboard OrderDetailModal: mobile bottom-sheet vs desktop centered modal.
- 261. GET /api/menu/items (list with optional category/search filter).
- 262. GET /api/menu/items/[slug] (single item with all images).
- 263. GET /api/menu/categories (list with item counts).
- 264. GET /api/menu/featured (homepage rail).
- 265. POST /api/checkout (create order with 503 paused, 401 unauth, 400 invalid, 404 address not found, 5-retry order-number generation).
- 266. GET /api/orders (customer history, last 50).
- 267. POST /api/orders/[id]/rate (1-5 star + review, only for DELIVERED, only once).
- 268. GET /api/orders/[id]/receipt (PDF download, public via unguessable CUID).
- 269. POST /api/orders/[id]/upi/initiate (start UPI attempt, 5-min window, expires prior attempts).
- 270. POST /api/orders/[id]/upi/upload (upload screenshot + Gemini verify + auto-approve if all checks pass).
- 271. GET /api/orders/[id]/upi/status (poll + auto-expire on read).
- 272. POST /api/orders/[id]/upi/manual (continue without screenshot).
- 273. GET /api/addresses (list, graceful for unauth).
- 274. POST /api/addresses (create, auto-default if first).
- 275. PATCH /api/addresses/[id] (update, isDefault clears others).
- 276. DELETE /api/addresses/[id] (delete, ownership check).
- 277. GET /api/settings (public, only isAcceptingOrders, fail-open on DB error).
- 278. GET /api/auth/me (profile auto-create + sync from Supabase).
- 279. PATCH /api/auth/me (update name/phone + sync to Supabase user_metadata).
- 280. GET /api/ratings (public aggregate).
- 281. POST /api/telegram/webhook (handle callback_query verify/reject).
- 282. GET /api/telegram/webhook (health-check).
- 283. POST /api/admin/login (hardcoded credential check, sets ab_admin=1 cookie).

- 284. GET /api/admin/orders (list with status/upiStatus/date filters).
- 285. PATCH /api/admin/orders/[id] (update status + payment received).
- 286. DELETE /api/admin/orders/[id] (permanent delete).
- 287. PATCH /api/admin/orders/[id]/upi-review (admin approve/reject).
- 288. GET /api/admin/settings (full config).
- 289. PATCH /api/admin/settings (toggle isAcceptingOrders).
- 290. GET /api/admin/export?type=orders|revenue (CSV export).
- 291. GET /api/admin/reviews (list + aggregate stats).
- 292. GET /api/admin/menu/items (list ALL including unavailable + featured).
- 293. POST /api/admin/menu/items (create with optional initial images).
- 294. GET /api/admin/menu/items/[id] (single item with all images).
- 295. PATCH /api/admin/menu/items/[id] (update + optionally REPLACE entire image set).
- 296. DELETE /api/admin/menu/items/[id] (delete + remove Supabase Storage images).
- 297. POST /api/admin/menu/items/[id]/images (upload single image, ≤5MB, count < 5).
- 298. GET /api/admin/menu/featured (list all featured).
- 299. PATCH /api/admin/menu/featured (toggle mode OR reorder mode).
- 300. GET /api/admin/menu/categories (list ALL including inactive).
- 301. POST /api/admin/menu/categories (create).
- 302. PATCH /api/admin/menu/categories/[id] (update).
- 303. DELETE /api/admin/menu/categories/[id] (refuse if items assigned).
- 304. POST /api/admin/menu/categories/[id]/icon (upload category icon).
- 305. GET /api/admin/stats (today + total + active + counts).
- 306. GET /api/admin/stats/sales (7-day chart + top 5 items last 30 days).
- 307. Category table (id, name, slug, iconUrl, description, sortOrder, isActive, timestamps).
- 308. MenuItem table (id, name, slug, description, price, categoryId, isAvailable, isVeg, prepTimeMins, calories, rating, sortOrder, isFeatured, featuredOrder, timestamps).
- 309. MenuItemImage table (id, itemId, url, sortOrder, createdAt).
- 310. Customer table (id, supabaseUserId, email, name, phone, avatarUrl, timestamps).
- 311. Address table (id, customerId, label, houseFlat, streetArea, landmark, city, pincode, latitude, longitude, distanceKm, isDefault, timestamps).
- 312. Order table (id, orderNumber, customerId, addressId, snapshot fields, status, paymentMode, paymentStatus, paymentReceived, paymentReceivedMethod, upiStatus, verifiedUtr, upiScreenshotUrl, upiVerificationResult, telegramMessageIds, bill snapshot fields, distanceKm, notes, rating, review, timestamps).
- 313. OrderItem table (id, orderId, itemId nullable, itemName, itemPrice, quantity, imageUrl, createdAt).
- 314. OrderStatusHistory table (id, orderId, status, note, createdAt).
- 315. PaymentAttempt table (id, orderId, attemptNumber, status, upiTxnRef, payeeVpa, payeeName, amount, startedAt, expiresAt, screenshotUrl, uploadedAt, gemini fields, failedChecks, verifiedAt, admin review fields, timestamps).
- 316. RestaurantConfig table (singleton id=1, isAcceptingOrders, updatedAt).
- 317. Turso (libSQL) primary database via Prisma + @prisma/adapters-libsql.
- 318. Supabase Auth for customer authentication (cookie-persisted session, Google OAuth, email verification).
- 319. Supabase Storage for image storage (2 buckets: menu-items + media).
- 320. Telegram Bot API for admin notifications + inline admin actions (verify/reject).
- 321. Google Gemini Vision (gemini-2.5-flash) for UPI screenshot verification.
- 322. OpenStreetMap Nominatim for geocoding + reverse-geocoding.
- 323. Leaflet + react-leaflet for interactive map (OSM tiles, custom SVG markers, draggable pin).
- 324. Google Maps deep-links (View on map + Get directions) — no API key required.
- 325. pdf-lib + @pdf-lib/fontkit for server-side PDF receipt generation.
- 326. Prisma ORM with driverAdapters preview feature.
- 327. Zustand with persist middleware (localStorage) for client state.
- 328. qrcode package for UPI deep-link QR code generation.
- 329. Recharts for 7-day revenue bar chart.
- 330. @dnd-kit for featured items drag-and-drop reorder.
- 331. Framer Motion for animations throughout.
- 332. Vercel runtime target (output: 'standalone', env var detection for self-CDN fetch).
- 333. Caddy reverse proxy on :81 with dynamic-port routing.

- 334. Customer auth soft-gating (browse without login; login required for favorites, checkout, address save, orders view).
- 335. Admin auth (hardcoded env-var credentials + literal '1' cookie, 12h maxAge, httpOnly, sameSite strict).
- 336. Phone-required guard (checkout forces profile update if no phone).
- 337. Reactive cache invalidation across admin + public query keys after every mutation.
- 338. Live delivery-fee calculation (client-side mirrors server).
- 339. Live distance badge + Within/Outside chip.
- 340. Live reverse geocoding (debounced 500ms).
- 341. Live UPI countdown timer.
- 342. Live order-acceptance polling (30s customer, 15s admin).
- 343. Live sales analytics refresh (60s).
- 344. Live new-order toasts (15s admin).
- 345. Optimistic UI updates (drag-reorder, status changes, payment updates).
- 346. Auto-expire UPI attempts on read.
- 347. Order-tracking timeline animation.
- 348. Multi-chat Telegram message sync (editAllTelegramMessagesForOrder).
- 349. Telegram inline keyboard with Verify/Reject + Map/Directions URL buttons.
- 350. Telegram webhook secret validation (X-Telegram-Bot-API-Secret-Token header).
- 351. Restaurant acceptance toggle (isAcceptingOrders).
- 352. Menu item availability toggle.
- 353. Category visibility toggle.
- 354. Featured toggle (isFeatured).
- 355. Featured order (featuredOrder, drag-and-drop reorder).
- 356. Order status override (5 statuses).
- 357. Payment-received marking (Cash/UPI at door).
- 358. UPI payment approve/reject (admin web UI + Telegram button).
- 359. Bulk order status update (admin Orders tab).
- 360. Pricing engine (10km radius, ₹20-₹70 delivery fee, ₹200/₹800 minimum order tiers, ₹2000 free-delivery override, round-to-nearest-5).
- 361. Quantity clamping (1-50).
- 362. Order number generation (AB-YYYY-NNNN via MAX+1, 5 retries for P2002).
- 363. Address auto-default (first address).
- 364. Address isDefault clear-then-set pattern.
- 365. Address ownership check on all mutations.
- 366. Receipt PDF generation (A4, brand letterhead, Poppins fonts, ☒ glyph, multi-page support).
- 367. Receipt download via blob + object URL + programmatic <a download>.click().
- 368. Image upload validation (image type, ≤5MB).
- 369. Image upload path conventions (menu-items/{slug-or-id}/, categories/{slug-or-id}/, payment-screenshots/{orderId}/).
- 370. Image gallery (single + multi-image carousel + Lightbox).
- 371. Veg indicator (square border + dot, emerald-600).
- 372. Star rating display (5 stars, amber filled).
- 373. Hardware back-button integration (customer + admin).
- 374. Bottom-sheet modals on mobile / centered modals on desktop.
- 375. Responsive grids (featured 2/3/4/5/6, popular 2/3/4/5/6, category 4/6/8 home / 2/3/4/6 view).
- 376. Safe area insets (iPhone notch support).
- 377. useIsMobile hook (matchMedia max-width: 767px).
- 378. Custom Leaflet SVG markers (red pin + black storefront).
- 379. Live distance label updates as pin drags.
- 380. Auto-locate on mount (geolocation).
- 381. Reverse-geocode auto-fill (pincode + city).
- 382. Search-as-you-type with results dropdown.
- 383. AuthedFetch helper (transparently attaches Supabase Bearer token).
- 384. Auth state sync (Customer ↔ Supabase user, email-based fallback merge).
- 385. Update user_metadata sync to Supabase Admin API.
- 386. Three Zustand stores with persist (cart, favorites, recent searches).
- 387. App store with partialize persistence (only selectedAddressId, activeOrder, upiConfirmationStatus).

- 388. Stack-trim on order confirmation (removes checkout + upi-payment from stack).
- 389. 11 maintenance scripts in /scripts (diagnostic, migration, verification).
- 390. Prisma schema-first ORM with driverAdapters preview feature.
- 391. Singleton Prisma client cached on globalThis in non-production.
- 392. next.config.ts with output: 'standalone', reactStrictMode: false, allowedDevOrigins for sandbox preview, single Supabase remote pattern for images.
- 393. Build script copies static + public into standalone bundle.

Total distinct features catalogued: 393. This count represents the complete, deduplicated feature surface of the Apna Baithak production codebase as of the audit date. Each item above is a single, countable capability that would need to be re-implemented in a rebuild scenario. The audit deliberately ignores bugs, code smells, and security observations; the only goal is a complete and accurate count of capabilities for development-cost estimation.